

Chapter 72B · Data Cleaning & Wrangling Question Bank

New chapter, 100 questions. Every number measured against companion/ch14/ and Chapter 14's clean truth file. 3 October 2026.

Chapter 72B. Data Cleaning & Wrangling Question Bank

Part 8 — Be Interview Ready

> **Chapter at a glance** > > **You will practise:** answering the cleaning questions that decide most data-role interviews, because cleaning is where the job actually is · profiling an unfamiliar file before you touch a single value · proving a row count rather than trusting one · parsing four date formats in one column without silently destroying two thirds of it · standardising categories with a mapping table instead of a pile of `.replace()` calls · spotting numbers that are text, discounts that are fractions, and quantities in the wrong unit · repairing a join key and measuring the match rate before you join · writing validation rules that must return zero · reconciling a cleaned table to its source to the rupee, and keeping a log someone else can audit. > > **Before you start: do Chapter 14 first** (Cleaning & Wrangling Real Data). This bank is the interview practice for that chapter's method, and it uses the same files. You also need Chapter 17 (Python from zero), Chapter 25 (pandas), and Chapter 69 (the three answer tiers and the twelve extra-point tags). Chapter 72's pandas gotchas come back here three times. > > **Time needed:** 7–9 hours to work through with the files open, plus 1 hour for a revision pass. Sections 72B.4 and 72B.7 are the two that most often decide an interview; give each its own sitting. > > **How this chapter is built.** Same format as the other question banks in Part 8: every core question leads with a "**Remember it as...**" hook, then a one-line answer, then a compact tier table (what **passes**, what's **strong**, and the **extra points**, tagged with Chapter 69's moves). Rapid-fire sections are scan tables. > > **What makes this bank different: every number in it was measured.** The questions run against two real files that ship with the book —

companion/ch14/orders_q4_2025_export.csv and companion/ch14/customers_crm_export.csv — and Chapter 14 also ships `clean_truth_orders_q4_2025.csv`, the correct answer. So when a question says a mistake costs ₹1.95 crore, that figure came from running both versions and subtracting, not from an estimate. **Every output shown is real**, on Python 3.12.0 with pandas 3.0.2 and numpy 2.4.3. Timings are not quoted anywhere in this chapter, because none of its points depend on speed. You can reproduce every single figure; §72B.11 tells you how.

72B.1 Why this is the round that matters

Ask any working analyst how their week divides and the answer is some version of "most of it was getting the data into a usable state." Job ads know this: "data cleaning" appears in nearly every one. And yet it is the weakest part of almost every candidate's preparation, because cleaning is unglamorous, it does not demo well, and no portfolio project ever says "here is the four hours I spent discovering that one branch writes dates backwards."

That gap is the opportunity. An interviewer who asks a cleaning question is rarely testing whether you know `.drop_duplicates()`. They are testing three things:

1. **Do you look before you act?** The candidate who starts writing transformations before profiling the file is the candidate who will silently corrupt a report.
2. **Do you know that a cleaning step can be wrong?**

`pd.to_datetime(col, errors='coerce')` is one line, looks responsible, and on the file in this chapter it destroys 17,090 of 25,970 dates without raising anything. Knowing that is the difference between a junior and a mid-level answer. 3. **Can you prove the result?** Not "I cleaned it" but "here are the four checks that return zero, and here is the reconciliation to the source total."

The third one is what almost nobody does, and it is the single highest-leverage thing in this chapter.

The file this bank uses. Riverstone Foods exported its October–December 2025 order lines from the ERP, with each branch sales office's data-entry habits left intact. Chapter 14, section 14.2 describes it in full. For this bank, what matters is the shape of the damage, which is a fair sample of what a real export looks like:

	In orders_q4_2025_export.csv
Rows below the header	25,976, of which 25,832 are genuine order lines
Junk rows	6 repeated header rows, 1 footer row
Duplicates	137 planted duplicate lines
Date formats	4 in one column, plus 9 dates that do not exist
status	18 spellings for 4 real statuses
branch	17 spellings for 4 real branches
unit_price	2,316 rows written as currency text (Rs . 430 , ₹1,400.00)
discount_pct	Written as a percent on most rows and as a fraction on 1,362
quantity	141 rows in cartons instead of pieces, 6 with a typed extra zero, 14 blank
customer_code	537 rows that lost a leading zero
Timestamps	entered_at_utc is UTC; on 1,598 rows its date is not the Indian date

Every one of those is a question in this chapter, and every one of them has a measured cost.

Levels and roles. Each question carries a level and the roles that usually ask it:

- **Fresher:** screening calls and first-job interviews. **Mid:** one to three years in the role. **Senior:** lead or specialist rounds.
- **DE** data engineer · **AE** analytics engineer · **MLE** machine learning engineer · **DS** data scientist · **DA** data analyst · **BA** business analyst.

Cleaning questions are asked of **every** role on that list, which is not true of any other bank in this part. A DSA round skips the analysts; a product-sense round skips the engineers. Cleaning skips nobody.

72B.2 Profiling: what to do before you change anything

Q72B-001 · A stakeholder sends you a CSV you have never seen. What are the first five things you do?

Level: Fresher · **Roles:** DA, DS, DE, AE, BA

Remember it as: *Look, count, and shape — before you fix. Every minute of profiling saves an hour of debugging a wrong number.*

Answer in one line: Before any transformation: check the file's physical shape (rows, columns, encoding, delimiter), look at actual raw values rather than types, count distinct values and nulls per column, check ranges and formats on the columns that matter, and only then decide what cleaning is needed — writing down what you found as you go.

The five, in order, with the reason each one exists:

	What you do	What it catches
1	Read the first and last few raw lines as text , not as a DataFrame	Headers that are not on line 1, footers, a totals row, the wrong delimiter, a BOM
2	Load everything as string and count rows	Type guessing that has already corrupted a value before you ever saw it
3	<code>nunique()</code> and blank counts per column	Categories with 18 spellings, columns that are entirely empty, keys that are not unique
4	Look at the actual distinct values of every low-cardinality column	Exactly the D1vd / Delivered / DELIVERED problem
5	Range and format checks on dates, money and quantities	Impossible dates, negative prices, a quantity of 700 where the median is 35

Step 2 is the one candidates skip, and it is the most important. Loading as string is how you see the damage before pandas hides it:

```
import pandas as pd

raw = pd.read_csv('orders_q4_2025_export.csv', dtype=str, keep_default_na=False)
print(raw.shape)
print(sorted(raw['unit_price'].unique()))
```

```
(25976, 13)
['', '115', '1400', '290', '380', '430', '620', '750', 'Rs. 1,400', 'Rs. 115',
 'Rs. 290', 'Rs. 380', 'Rs. 430', 'Rs. 620', 'Rs. 750', 'unit_price',
 '₹1,400.00', '₹115.00', '₹290.00', '₹380.00', '₹430.00', '₹620.00', '₹750.00']
```

Thirteen columns, 25,976 rows, and one `print` has told you four separate things: `unit_price` is not a number, there are **seven real prices written twenty-one ways**, there is a blank, and the literal text `unit_price` is sitting in the data — which means the file contains copies of its own header row. Load that same file with pandas' defaults and `unit_price` comes back as `object` with no explanation, while a careless `pd.to_numeric` later turns 2,316 real prices into `NaN`. Chapter 14 makes the same point about the SQL side: the staging table is deliberately all `TEXT`, so `31-11-2025` arrives unchanged instead of being rejected or silently converted.

Tier	What to say
Passes	<code>.head()</code> , <code>.info()</code> , <code>.describe()</code> , <code>.isnull().sum()</code> — the right instinct, but all four read what pandas <i>inferred</i> , not what the file <i>contains</i>
Strong	The five steps above, with string loading named explicitly, and the reason: profiling must see the raw bytes, because type inference is itself a transformation that can lose data
Extra points	+ [+Validate] say that you write the profile down as numbers (row count, distinct count per column) so that after cleaning you can prove what changed + [+Clarify] ask what the file is <i>for</i> before profiling, because the columns that matter decide which checks are worth the time + [+Business] ask who produced it and whether the process is manual, which predicts the kind of damage you will find

Likely follow-ups: How would you profile a file too large to fit in memory? What would you do differently if this were a database table instead of a CSV?

Red flag: starting to write cleaning code in the first minute, before describing anything about the data.

Learn it in: Chapter 14, §14.3 (the profiling pass) and §14.2 (the files); Chapter 25, §25.4 (reading with `dtype=str`).

Q72B-002 · Why load every column as a string first, when pandas can infer types for you?

Level: Mid · **Roles:** DA, DS, DE, AE

Remember it as: *Type inference is a transformation, and it runs before you have seen the data. String loading is the only way to see the file as it actually is.*

Answer in one line: Because inference makes irreversible decisions — dropping leading zeros from codes, guessing a date format from the first rows it happens to see, turning a mixed column into `object` with no warning — and once those decisions are made, the original value is gone and you cannot audit what happened.

Measured, on this file — and the result is not what most candidates expect. Read it with pandas' defaults and look at the dtypes:

```
default = pd.read_csv('orders_q4_2025_export.csv')
print(default.dtypes.to_string())
print('customer_code:', default['customer_code'].head(5).tolist())
```

```
order_item_id    str
order_id         str
order_date       str
customer_code     str
product_id       str
quantity         str
qty_unit         str
unit_price       str
discount_pct     str
status           str
sales_rep        str
branch           str
entered_at_utc   str

customer_code: ['0124', '0124', '0153', '0153', '0161']
```

Every column came back as `str`, and the leading zeros survived. On pandas 3.0 that is partly the new string default, but on this file there is a second reason and it is the interesting one: the 6 repeated header rows put the text `customer_code` inside the `customer_code` column, so no column *can* be inferred as numeric. **The junk is accidentally protecting the data.**

Now do the obviously correct thing — remove the junk rows first — and read it again:

```
body = raw[(raw['order_item_id'] != 'order_item_id') & (raw['order_item_id'] != '')]
body.to_csv('body_no_junk.csv', index=False)

cleaned = pd.read_csv('body_no_junk.csv')
print(cleaned[['order_item_id', 'customer_code', 'product_id', 'quantity']].dtypes.to_string())
print('customer_code:', cleaned['customer_code'].head(5).tolist())
```

```
order_item_id      int64
customer_code      int64
product_id         float64
quantity           float64

customer_code: [124, 124, 153, 153, 161]
```

You did the right thing and it broke the data. With the text gone, inference now works, `customer_code` becomes `int64`, and `0124` becomes `124` — silently losing the join key on 537 rows carrying ₹91.7 lakh (Q72B-057 prices it exactly). Two more things went wrong in the same read: `product_id` and `quantity` came back as `float64` rather than integers, because 8 and 14 blank values respectively force the column to hold `NaN`, so a product ID now prints as `101.0`.

This is the strongest argument for string loading there is, and it is worth saying in exactly this shape: **the safety of the default read depended on the file being dirty**. Fix one problem and the next one appears. An explicit `dtype` is the only thing that does not change behaviour when the data does.

Tier	What to say
Passes	"To avoid pandas guessing wrong," correct but without naming a specific loss
Strong	Names the losses — leading zeros, integers becoming floats because of blanks, a guessed date format — and the principle: inference is a transformation, so it belongs <i>after</i> profiling, not before
Extra points	+ [+Validate] the demonstration above, which is the strongest form of this answer: inference changed behaviour <i>because</i> a cleaning step succeeded, so the default read was only ever safe by accident + [+Edge cases] <code>keep_default_na=False</code> matters as much as <code>dtype=str</code> : without it pandas still converts the literal text <code>N/A</code> , <code>NULL</code> and <code>NA</code> to <code>NaN</code> , so you cannot tell a blank cell from a typed "N/A", which are different problems with different fixes + [+Trade-offs] string loading costs memory and speed, so on a very large file you profile a sample as strings and then load the full file with an explicit <code>dtype</code> dict — which is the real goal, since the output of profiling is that dict

Likely follow-ups: What does `low_memory=False` actually do? Once you have profiled, what does the ideal `read_csv` call look like? Why did `product_id` become a float?

Red flag: not knowing that `read_csv` converts the text `N/A` to `NaN` by default; or assuming inference behaves identically across pandas versions, when the string default changed in pandas 3.0.

Learn it in: Chapter 14, §14.3; Chapter 25, §25.4 and §25.5 (dtypes and the `na_values` family).

Q72B-003 · How do you tell whether a column that looks numeric actually is?

Level: Fresher · **Roles:** DA, DS, DE, AE

Remember it as: *Don't ask the dtype, ask the values. object tells you something failed; it never tells you what.*

Answer in one line: Load it as string and test every value against the pattern you expect, counting and showing the ones that fail — rather than reading the dtype, which only tells you that inference gave up, or using `to_numeric` with `errors='coerce'`, which hides the failures as `NaN`.

Measured. `unit_price` on this file has only 22 distinct values, which is itself the fastest clue:

```
price = asstr['unit_price']
print('distinct values:', price.nunique())

bad = price.str.contains(r'[^\d-9.]', regex=True) & (price != '')
print('values with a non-numeric character:', f'{bad.sum():,} rows')
print(sorted(set(price[bad]))[:8])
print('the real prices underneath:', sorted(set(price[~bad & (price != '')])))
```

```
distinct values: 22
values with a non-numeric character: 2,316 rows
['Rs. 1,400', 'Rs. 115', 'Rs. 290', 'Rs. 380', 'Rs. 430', 'Rs. 620', 'Rs. 750', '₹1,400.00']
the real prices underneath: ['115', '1400', '290', '380', '430', '620', '750']
```

Seven real prices, written 22 ways. Note what the count tells you: a price column with 22 distinct values across 25,976 rows is a *catalogue* price, not a negotiated one, which is a useful business fact you got for free while profiling.

Tier	What to say
Passes	<code>pd.to_numeric(col, errors='coerce').isna().sum()</code> to count the failures — a reasonable count, but it does not show you <i>what</i> failed, so you cannot design the fix
Strong	The pattern test above, showing the failing values, and the point that you need to see them to know whether the fix is "strip currency symbols" or "this column has two different units in it"
Extra points	+ [+Validate] after the fix, assert that zero non-null values fail the pattern, and that the distinct count dropped from 22 to 7 + [+Edge cases] name the traps the pattern must handle: thousands separators, a trailing minus or parentheses for negatives, a Unicode minus sign, and a value like 1.400,00 in European format where the comma is the decimal

Likely follow-ups: Write the expression that cleans ₹1,400.00 and Rs. 1,400 to 1400.0. How would you handle a column with both 1,400.00 and 1.400,00 in it?

Red flag: `astype(float)` with no check, which raises on the first currency string and tells you nothing about the other 2,315.

Learn it in: Chapter 14, §14.9 (units and currency text); Chapter 25, §25.7.

Q72B-004 · What does a good profile report contain, and why write it down?

Level: Mid · **Roles:** DA, AE, BA

Remember it as: *A profile is a before-photograph. Without one you can clean a file but you can never prove what you changed.*

Answer in one line: Per column: the dtype as loaded, distinct count, blank and null count, the actual distinct values if cardinality is low, min/max or range for dates and numbers, and the pattern violations — plus the file-level row count; written down because every one of those numbers becomes a reconciliation check after cleaning.

The link between profiling and proof. This is the part most candidates miss. The profile is not documentation for its own sake; each line of it is a *test you can run again*:

Profiled before	Becomes this check after
25,976 rows in file	25,832 clean rows + 137 duplicates + 6 headers + 1 footer = 25,976, exactly
<code>status</code> has 18 distinct values	<code>status</code> has exactly 4, and zero nulls
<code>unit_price</code> has 22 distinct values	<code>unit_price</code> has 7, all numeric
2,316 rows of currency text	zero rows failing the numeric pattern

The left column is profiling; the right column is the validation suite in §72B.9. **They are the same list.** That is the insight worth saying out loud in an interview, because it reframes profiling from a chore into the thing that makes the result defensible.

Tier	What to say
Passes	Lists reasonable contents of a profile
Strong	The contents plus the observation that each profiled number becomes a post-cleaning check, so profiling and validation are one activity done at two times
Extra points	+ [+Validate] the row-count arithmetic above: every row is accounted for as kept, de-duplicated or quarantined, and the four numbers sum to the original + [+Business] a written profile is what lets you tell a stakeholder "1,362 of your discount values were recorded as fractions" — which usually fixes the upstream form, and is worth more than the cleaning

Likely follow-ups: Would you use a tool like `ydata-profiling` for this? What does it not tell you?

Red flag: treating profiling output as something you glance at and discard.

Learn it in: Chapter 14, §14.3 and §14.14 (the cleaning log).

Q72B-005 · Your file has 25,976 rows. How many orders is that?

Level: Fresher · **Roles:** DA, DS, BA, AE

Remember it as: *A row is not a thing. Always ask what one row means before you count anything.*

Answer in one line: Unknown until you check the grain: this file is one row per **order line**, so 25,976 rows is neither 25,976 orders nor 25,976 anything else — it is 25,832 genuine lines belonging to 14,372 orders, once the junk and duplicates are removed.

Measured, the whole chain:

```
d = raw[raw['order_item_id'] != 'order_item_id']      # drop 6 repeated headers
d = d[d['order_item_id'] != '']                     # drop 1 footer row
d = d.drop_duplicates(subset=['order_item_id'])      # drop 137 duplicates

print(f'genuine order lines : {len(d):,}')
print(f'distinct orders      : {d["order_id"].nunique():,}')
print(f'lines per order      : {len(d) / d["order_id"].nunique():.2f}')
```

```
genuine order lines : 25,832
distinct orders      : 14,372
lines per order      : 1.80
```

Why this question is asked so often. Getting the grain wrong is the most common cause of a confidently wrong number in business reporting, and it is invisible: a dashboard that says "25,976 orders this quarter" when the answer is 14,372 looks perfectly fine. It is off by 81%.

Tier	What to say
Passes	"I would check whether there are duplicates"
Strong	Names grain as the first question, establishes it is one row per order line, and distinguishes the three different numbers: rows in file, genuine lines, distinct orders
Extra points	+ [+Clarify] ask what the stakeholder means by "order", because an order with three lines is one order to finance and three picks to the warehouse — both are right for their purpose + [+Validate] 1.80 lines per order is a sanity check in itself: a value of 1.00 would suggest the file is actually order-level, and a value of 20 would suggest something is duplicating + [+Edge cases] note that <code>nunique()</code> on <code>order_id</code> is only trustworthy <i>after</i> the junk rows are gone, since the repeated header rows contribute the literal string <code>order_id</code> as a value

Likely follow-ups: How would you report "average order value" from this file? What is the grain of the customer file?

Red flag: answering "25,976" with no qualification.

Learn it in: Chapter 14, §14.4; Chapter 28, §28.4 (grain in a data model); Chapter 10, §10.3.

Rapid-fire, 72B.2

Roles: DA, DS, DE, AE and BA for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q72B-006	What is a BOM and how does it break a CSV read?	A byte-order mark is three invisible bytes some tools write at the start of a UTF-8 file; read without <code>encoding='utf-8-sig'</code> the first column name silently becomes <code>column</code> instead of <code>column</code> , so every reference to it raises <code>KeyError</code>	[+Edge cases] the symptom is a <code>KeyError</code> on a column you can plainly see in the header, which is why it wastes so much time	Fresher · 14.3
Q72B-007	How do you detect the delimiter of an unfamiliar file?	Read the first few lines as text and look, or let <code>csv.Sniffer</code> guess; never assume a comma, because exports from Indian and European systems frequently use semicolons so that commas inside numbers survive	[+Validate] the giveaway of a wrong delimiter is a <code>DataFrame</code> with one column whose name contains every header	Fresher · 14.3
Q72B-008	A column has 64 distinct values and should have about 40. What is going on?	Almost certainly category variants — case, whitespace, abbreviations, or old names for the same thing — which is exactly the <code>city</code> column in this book's CRM file, where 64 distinct values map down to 39 real cities	[+Business] the ratio of distinct values to expected values is the fastest single profiling metric for a categorical column	Fresher · 14.8
Q72B-009	Why check <code>min</code> and <code>max</code> on a date column during profiling?	Because they catch the two most common date disasters in one step: a parse that silently produced dates in 1970 or 2106, and genuine out-of-range data such as the CRM's 5 signup dates in the 2060s	[+Edge cases] a max date far in the future usually means a placeholder like <code>9999-12-31</code> used as "no end date"	Fresher · 14.7
Q72B-010	What does <code>.describe()</code> not tell you?	Anything about the columns that matter most: it silently skips every non-numeric column by default, says nothing about nulls as a proportion, and reports a mean and standard deviation for ID columns where both are meaningless	[+Trade-offs] <code>.describe(include='all')</code> covers more but makes the output wide enough to be unreadable, which is why targeted checks beat one summary call	Fresher · 25.6

72B.3 The row count: duplicates, headers and footers

Q72B-011 · Your 25,976-row file contains 6 copies of its own header row. How did they get there, and how do you remove them safely?

Level: Fresher · **Roles:** DA, DE, AE

Remember it as: *Repeated headers mean the export was paginated. Filter them by value, never by position.*

Answer in one line: They come from a paginated report export, where the header is re-printed at the top of each page and the pages were concatenated; remove them by filtering rows whose key column equals the column's own

name, not by dropping fixed row numbers, which breaks the moment the page size changes.

Measured:

```
hdr = raw['order_item_id'] == 'order_item_id'
print(f'repeated header rows: {hdr.sum()}')
print(f'at file positions : {raw.index[hdr].tolist()}')
```

```
repeated header rows: 6
at file positions : [4028, 8047, 12061, 16090, 20109, 24130]
```

The positions give the game away: gaps of 4,019, 4,014, 4,029, 4,019 and 4,021 rows. Near-regular but not identical, which is the signature of **pagination at roughly a 4,000-row page size** with extra rows interleaved — here, the 137 planted duplicates. The exact page size is an inference rather than a measurement, but the pagination is not in doubt. **That is worth saying out loud**, because it means this recurs on every future export, so the fix belongs in the pipeline permanently, not in a one-off notebook.

Why filtering by value and not position. `raw.drop([4028, 8047, ...])` works today and silently corrupts next month's file when the page size changes or a page is partially full. The value filter is correct for any page size:

```
clean = raw[raw['order_item_id'] != 'order_item_id']
print(f'{len(raw):,} -> {len(clean):,}')
```

```
25,976 -> 25,970
```

Tier	What to say
Passes	Spots them and filters them out by value
Strong	The value filter plus the diagnosis from the positions: a 4,000-row page size, so this is a recurring structural property of the source, not a one-off accident
Extra points	+ [+Validate] assert the removed count is exactly the number you profiled, so a change in the source announces itself as a failed assertion rather than a quiet drift + [+Business] the real fix is upstream: ask for a raw data export instead of a paginated report, which removes this problem and the footer problem together + [+Edge cases] a header row survives most type coercions as a string, so if you load with inference first, these 6 rows turn a numeric column into <code>object</code> and you will chase the wrong bug

Likely follow-ups: What if the repeated header had slightly different capitalisation each time? How would you handle this in a SQL staging table?

Red flag: dropping rows by index position.

Learn it in: Chapter 14, §14.4; Chapter 45, §45.8 (schema drift in an automated pipeline).

Q72B-012 · The last row of the file has a blank ID and a total in the quantity column. What is it, and what happens if you miss it?

Level: Fresher · **Roles:** DA, DE, AE, BA

Remember it as: A footer row is a number that double-counts itself. It inflates every total by exactly 100%.

Answer in one line: It is a report footer carrying column totals, which must be removed before any aggregation, because leaving it in adds the entire file's total to the file's total — doubling every sum while leaving counts off by only one, so the error looks like a data problem rather than a structural one.

Measured. The footer on this file is the single row with a blank `order_item_id`:

```
foot = clean[clean['order_item_id'] == '']
print(foot[['order_item_id', 'order_id', 'order_date', 'quantity']].to_string(index=False))
```

```
order_item_id order_id order_date quantity
```

Blank across the board on this particular export, which is the easy case. The dangerous case is a footer that carries real totals, and the reason it is dangerous is the asymmetry: a `COUNT` goes from 25,832 to 25,833, which nobody notices, while a `SUM` doubles, which everybody notices but nobody attributes to one extra row.

Tier	What to say
Passes	Removes the row because the ID is blank
Strong	Names it as a report footer, removes it by the structural rule ("no primary key, therefore not a fact"), and explains the count-versus-sum asymmetry that makes it hard to diagnose
Extra points	+ [+Validate] the general rule that generalises past this one file: every fact row must have a non-null primary key , so quarantine anything without one and look at it, rather than writing a rule about blank quantity + [+Edge cases] a footer sometimes also appears as the <i>first</i> row ("Report generated on..."), so check both ends + [+Business] same upstream fix as Q72B-011: a data export has no footers because it is not a report

Likely follow-ups: How would you catch a footer whose ID was not blank but the text `Total`? What if the file had a footer per page?

Red flag: finding the footer only after a total looked wrong.

Learn it in: Chapter 14, §14.4; Chapter 10, §10.5 (why a report and a dataset are different things).

Q72B-013 · How do you find duplicate rows, and what makes a row a duplicate?

Level: Fresher · **Roles:** DA, DS, DE, AE

Remember it as: "Duplicate" is a business definition, not a pandas function. Decide the key first, then count.

Answer in one line: Define the key that is supposed to be unique, then count rows where it repeats — `duplicated(subset=key)` — because a fully identical row and a repeated business key are different problems, and the second is both more common and more dangerous.

Measured, on this file, three different definitions and three different answers:

```
print('identical across all 13 columns:', clean.duplicated().sum())
print('repeated order_item_id      : ', clean.duplicated(subset=['order_item_id']).sum())
print('repeated order_id          : ', clean.duplicated(subset=['order_id']).sum())
```

```
identical across all 13 columns: 137
repeated order_item_id      : 137
repeated order_id          : 11597
```

Read those three numbers carefully, because the gaps between them are the whole lesson.

- **137** on the first two lines, and they agree for a good reason: `order_item_id` is the primary key, so here a repeated key and a fully identical row are the same thing. Chapter 14's answer key confirms 137 planted duplicates. When those two counts *disagree*, you have a conflict rather than a duplicate — Q72B-019.
- **11,597** is not a data-quality problem at all. `order_id` *should* repeat, because the grain is order lines and an order averages 1.80 of them. A candidate who runs `drop_duplicates(subset=['order_id'])` here deletes 11,597 real order lines and destroys **44% of the revenue**: ₹46.01 crore of gross falls to ₹25.71 crore.

That third number is the trap, and it is the reason the answer must start with "what is the key?" rather than with a function name.

Tier	What to say
Passes	<code>df.duplicated().sum()</code> and <code>drop_duplicates()</code>
Strong	Asks for the unique key first, distinguishes an exact duplicate from a repeated business key, and names the risk of de-duplicating on a column that is legitimately repeated
Extra points	+ [+Validate] before de-duplicating on a key, check that the duplicate rows really are identical in their other columns; if they differ, you have a conflict to resolve, not a duplicate to drop + [+Edge cases] <code>keep='first'</code> is a decision, not a default to accept: if the rows differ, "first" means "whichever order the export happened to produce" + [+Business] quantify it before dropping, which here is 137 lines and the revenue they carry, so the stakeholder knows the size of the correction + [+Validate] compare the two counts — rows identical in every column, and rows with a repeated key — because their agreeing at 137 is itself evidence the key is behaving as a key

Likely follow-ups: The 137 duplicate rows differ in one column. Now what? How would you find *fuzzy* duplicates in the customer names?

Red flag: `drop_duplicates()` with no `subset` and no stated key, as the first action.

Learn it in: Chapter 14, §14.6 (exact and fuzzy duplicates); Chapter 25, §25.9.

Q72B-014 · Two customer records read "Riverstone Foods Pvt Ltd" and "riverstone foods pvt. ltd.". Are they duplicates, and how would you find all such pairs?

Level: Mid · **Roles:** DA, DS, AE, BA

Remember it as: *Normalise, then compare. Fuzzy matching is a last resort, not a first move, because it merges records that should not be merged.*

Answer in one line: Yes, under Riverstone's stated rule; find them by **normalising** both strings to a comparison key — trim, collapse internal spaces, lowercase, strip the legal suffix — and grouping on that key, resorting to similarity scoring only for the residue that normalisation cannot reach.

The rule, from Chapter 14, §14.15. Two customer records are duplicates if their names match after trimming, collapsing spaces, ignoring case, and removing a trailing "Pvt Ltd" (with or without dots) or "Private Limited". Note that this is a **business rule someone decided**, written down — not something a function infers. That is the point of the question.

```

cust = pd.read_csv('customers_crm_export.csv', dtype=str, keep_default_na=False)

key = (cust['customer_name'].str.strip()
       .str.replace(r'\s+', ' ', regex=True)
       .str.lower()
       .str.replace(r'\s*(pvt\.\s*|private\s+limited)$', '', regex=True)
       .str.strip())

print(f'records           : {len(cust):,}')
print(f'distinct name keys: {key.nunique():,}')
print(f'duplicate records : {key.duplicated().sum():,}')

```

```

records           : 5,027
distinct name keys: 4,979
duplicate records : 48

```

48, which matches Chapter 14's answer key exactly. **Normalisation alone found every planted duplicate, with no fuzzy matching at all** — and that is the honest headline. Deterministic normalisation is cheaper, explainable, reviewable and reproducible; similarity scoring is none of those things.

Tier	What to say
Passes	Suggests lowercasing and stripping, or jumps straight to a fuzzy-matching library
Strong	The normalisation chain with the business rule stated as a rule, the exact count found, and the judgement that fuzzy matching is for the residue only
Extra points	+ [+Validate] before merging anything, print the matched pairs for a human to review, because a merge is almost impossible to reverse once downstream data references the surviving ID + [+Edge cases] normalisation that is too aggressive creates false merges: "Riverstone Foods" and "Riverstone Foods South" are different companies, and stripping a trailing word would merge them + [+Trade-offs] name a threshold problem honestly — any similarity cutoff trades false merges against missed duplicates, and the right cutoff depends on which error is more expensive, which is a business question, not a technical one

Likely follow-ups: Which record survives a merge, and what happens to the orders pointing at the other one? How would you do this on 50 million records, where comparing every pair is impossible?

Red flag: reaching for `fuzzywuzzy` before trying normalisation, or auto-merging above a score with no human review.

Learn it in: Chapter 14, §14.6 and §14.15; Chapter 29, §29.7 (blocking, for the scale follow-up).

Q72B-015 · Walk me through accounting for all 25,976 rows, so that nothing is unexplained.

Level: Mid · **Roles:** DA, DE, AE, BA

Remember it as: *Every row leaves the pipeline through exactly one labelled door: kept, de-duplicated, structural, or quarantined. The doors must sum to the original count.*

Answer in one line: Build a reconciliation where the original row count equals the sum of rows kept, rows removed as duplicates, rows removed as structural junk, and rows quarantined — each category counted and labelled, so there is no residue and no row that silently disappeared.

Measured, and it closes exactly:

```
original = len(raw)
headers = (raw['order_item_id'] == 'order_item_id').sum()
footers = ((raw['order_item_id'] == '')).sum()
body = raw[(raw['order_item_id'] != 'order_item_id') & (raw['order_item_id'] != '')]
duplicates = body.duplicated(subset=['order_item_id']).sum()
kept = len(body) - duplicates

print(f' rows in file      {original:>8,}')
print(f' repeated headers {headers:>8,}')
print(f' footer rows        {footers:>8,}')
print(f' duplicate lines     {duplicates:>8,}')
print(f' genuine lines kept  {kept:>8,}')
print(f' -----')
print(f' sum of categories   {headers + footers + duplicates + kept:>8,}')
print(f' reconciles          {headers + footers + duplicates + kept == original}')
```

```

rows in file          25,976
repeated headers      6
footer rows           1
duplicate lines       137
genuine lines kept    25,832
-----
sum of categories     25,976
reconciles            True

```

This is the single most valuable habit in the chapter, and almost nobody does it. The reason it matters is that it converts "I cleaned the data" into an auditable statement. If a stakeholder asks why the report says 25,832 and their export says 25,976, the answer is a four-line table, not a shrug.

Tier	What to say
Passes	"I would note how many rows I dropped"
Strong	The full reconciliation above, with the categories summing to the original and the equality asserted rather than eyeballed
Extra points	+ [+Validate] make the final line an <code>assert</code> , so a future file that does not reconcile fails the pipeline instead of producing a quietly wrong report + [+Business] the four categories are exactly what a stakeholder or auditor wants to see, and "137 duplicate lines, here they are" closes a conversation that "the numbers differ" would open + [+Edge cases] quarantine is a fifth door and must be counted too: rows you neither kept nor deleted but set aside for review, which is where this file's 14 blank quantities and 6 extra-zero rows go

Likely follow-ups: Where do you store the quarantined rows, and who looks at them? What if a row belongs in two categories at once?

Red flag: a cleaning script whose output row count cannot be explained from its input row count.

Learn it in: Chapter 14, §14.13 (reconciling to source) and §14.14 (the cleaning log).

Rapid-fire, 72B.3

Roles: DA, DS, DE, AE and BA for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q72B-016	<code>drop_duplicates()</code> with no arguments — what is the risk?	It only removes rows identical in <i>every</i> column, so it misses the far more common repeated business key, and it silently picks the first occurrence when the rows differ in a column you did not check	[+Validate] always pass <code>subset=</code> explicitly, so the key is visible in the code and reviewable	Fresher · 14.6
Q72B-017	What is the difference between <code>keep='first'</code> , <code>keep='last'</code> and <code>keep=False</code> ?	<code>'first'</code> and <code>'last'</code> keep one occurrence and mark the rest as duplicates; <code>keep=False</code> marks every copy as duplicated, which is what you want for <i>inspecting</i> duplicates rather than removing them	[+Edge cases] <code>keep=False</code> is the right choice when you want to look at both sides of each duplicate pair before deciding	Fresher · 25.9
Q72B-018	Why does de-duplicating before parsing dates sometimes change the answer?	Because two rows holding the same date in different formats (<code>05-11-2025</code> and <code>2025-11-05</code>) are not identical as text but become identical after parsing, so the order of the two steps decides whether they are caught	[+Trade-offs] de-duplicate on a stable business key rather than on content, and the ordering stops mattering	Mid · 14.6
Q72B-019	A row repeats but one column differs. Duplicate or not?	Not a duplicate — a conflict , which needs a resolution rule (most recent wins, source priority, or escalate) rather than a silent drop that keeps whichever copy the export happened to list first	[+Business] this is the common case in CRM data, where two sales offices entered the same customer with different cities	Mid · 14.6
Q72B-020	How would you de-duplicate a stream, where you cannot see all the rows at once?	Keep a set of seen keys within a bounded window, or make the downstream write idempotent with an upsert on the business key so a duplicate overwrites rather than appends	[+Signpost] this is Q77-007's idempotency, which is the general answer to duplicates in any pipeline that can retry	Senior · 45.5, 50.3

72B.4 Dates: the column that breaks quietly

This is the most important section in the chapter. Dates are where cleaning goes wrong *silently* — the code runs, nothing raises, the null count looks fine, and the quarterly report is wrong by a third. Every figure below was measured on the 25,969 genuine-and-duplicate rows of `orders_q4_2025_export.csv` (the body of the file, after the 6 header rows and 1 footer row are removed).

Q72B-021 · One date column, four formats. What is in it, and how do you find out?

Level: Fresher · **Roles:** DA, DS, DE, AE

Remember it as: *Count the shapes before you parse. A format you did not know was there is a format you will parse wrongly.*

Answer in one line: Classify every value by its **shape** with regular expressions and count each group, so you know exactly which formats are present and in what proportion, before choosing how to parse any of them.

Measured:

```
s = body['order_date']

print('dd-mm-yyyy      ', s.str.match(r'^\d{2}-\d{2}-\d{4}$').sum())
print('yyyy-mm-dd      ', s.str.match(r'^\d{4}-\d{2}-\d{2}$').sum())
print('dd/mm/yyyy      ', s.str.match(r'^\d{2}/\d{2}/\d{4}$').sum())
print('5-digit serial  ', s.str.match(r'^\d{5}$').sum())
print('total           ', len(s))
```

```
dd-mm-yyyy      20890
yyyy-mm-dd      3227
dd/mm/yyyy      1812
5-digit serial   40
total           25969
```

20,890 + 3,227 + 1,812 + 40 = 25,969 exactly. That the four shapes account for every row is itself the most valuable output here: it means there is no fifth surprise format, so a parser handling these four is complete. If the sum had fallen short, the residue would be the first thing to look at.

Each shape also tells you something about its source. The `dd-mm-yyyy` majority is Indian data-entry convention; the ISO block is a system-generated export; `dd/mm/yyyy` is a second office with a different habit; and the 40 five-digit values are Excel serial numbers, which means at least one branch opened the file in Excel and saved it before sending.

Tier	What to say
Passes	<code>pd.to_datetime(col, errors='coerce')</code> and then counts how many failed
Strong	The shape census above <i>before</i> parsing, with the observation that the counts must sum to the row count for the parser to be known-complete
Extra points	+ [+Validate] the sum check, which proves there is no unknown fifth format + [+Business] read the formats as evidence about the sources: four formats means roughly four different entry habits, which is a process finding worth reporting upstream + [+Edge cases] note that a 5-digit value is an Excel serial and needs a completely different conversion from the other three, not a date format string

Likely follow-ups: What is the serial `45933` as a date? What would you do if the shapes did not sum to the row count?

Red flag: parsing first and profiling the failures afterwards, which only ever shows you the values that failed loudly.

Learn it in: Chapter 14, §14.7 (mixed date formats); Chapter 25, §25.8.

Q72B-022 • `pd.to_datetime(col, errors='coerce')` on this column. What happens?

Level: Mid • **Roles:** DA, DS, DE, AE

Remember it as: `errors='coerce'` does not make parsing safe. It makes failure silent.

Answer in one line: It destroys **17,089 of 25,969 dates** — 66% of the column — converting them to `NaT` without raising anything, because pandas picks one format from early rows and every value that does not match it becomes null.

Measured, four one-liners on the same column:

```
for kw in [{}, {'dayfirst': True}, {'format': 'mixed'}, {'format': 'mixed', 'dayfirst': True}]:
    nat = pd.to_datetime(s, errors='coerce', **kw).isna().sum()
    print(f'{str(kw) or "defaults":<40} {nat:>6} NaT')
```

```
defaults                                17089 NaT
{'dayfirst': True}                      5088 NaT
{'format': 'mixed'}                     49 NaT
{'format': 'mixed', 'dayfirst': True}   49 NaT
```

Two thirds of the column gone, and the only symptom is a null count that someone has to think to check. **A pipeline that does this and then reports "Q4 revenue" produces a number built from a third of the data.**

The pattern to notice is that each option improves the NaT count, which is exactly why this is dangerous: the obvious metric gets better while the data gets quietly worse. Q72B-023 shows that the last two options, identical at 49 NaT, disagree with each other about 1,330 dates.

Tier	What to say
Passes	"It would turn the unparseable ones into NaT" — correct in principle, with no sense of the scale
Strong	The measured 66% loss, and the mechanism: inference picks a single format and silently nulls everything else, so <code>errors='coerce'</code> converts a loud failure into a silent one
Extra points	+ [+Validate] always compare the null count <i>before</i> and <i>after</i> parsing and assert they are equal, because any increase is data you just destroyed + [+Trade-offs] <code>errors='raise'</code> is often the better default in a pipeline: a job that stops is cheaper than a report that is quietly wrong + [+Edge cases] <code>errors='coerce'</code> is right in exactly one situation — when you have already classified the shapes and are deliberately parsing one shape at a time (Q72B-024)

Likely follow-ups: So what is the correct way to parse this column? Why did `dayfirst=True` improve it so much?

Red flag: treating `errors='coerce'` as the responsible, defensive choice.

Learn it in: Chapter 14, §14.7; Chapter 25, §25.8.

Q72B-023 · You add `dayfirst=True` and the null count drops from 17,089 to 5,088. Have you fixed it?

Level: Senior · **Roles:** DA, DS, DE, AE

Remember it as: `dayfirst=True` fixes the Indian dates and breaks the ISO ones. There is no single flag that is right for a column with four formats.

Answer in one line: No — you have traded one error for a smaller but equally silent one, because `dayfirst=True` also applies to the 3,227 ISO `yyyy-mm-dd` values, where pandas swaps the last two components and turns 1 October into 10 January.

Measured. This is the finding worth remembering from the whole chapter:

```
iso = s.str.match(r'^\d{4}-\d{2}-\d{2}$')
wrong = pd.to_datetime(s[iso], errors='coerce', format='mixed', dayfirst=True)

print(pd.DataFrame({'raw': s[iso], 'parsed with dayfirst=True': wrong.dt.strftime('%Y-%m-%d')})
      .drop_duplicates('raw').head(6).to_string(index=False))
```

```

raw parsed with dayfirst=True
2025-10-01      2025-01-10
2025-10-02      2025-02-10
2025-10-03      2025-03-10
2025-11-01      2025-01-11
2025-11-02      2025-02-11
2025-11-03      2025-03-11

```

2025-10-01 is ISO 8601. It is unambiguous. `dayfirst=True` reverses it anyway, and **1,070 rows end up in the wrong month.**

So both one-liners are wrong in opposite directions, and neither announces it:

Parse	NaT	Rows it dates incorrectly
<code>format='mixed'</code>	49	8,804 (34.0%) — reverses the ambiguous <code>dd-mm</code> and <code>dd/mm</code> values
<code>format='mixed', dayfirst=True</code>	49	1,330 (5.1%) — reverses the ISO values

Both report exactly 49 nulls. That is the sentence to say out loud in an interview, because it means **no null check, no `isna()` count and no "did it parse" test can detect either bug.** The only thing that detects it is checking the *values*.

How bad it gets in a report. Same file, same gross revenue, grouped by month, in rupees crore:

Month	<code>format='mixed'</code>	correct parse
Jan–Sep 2025	₹12.78 cr spread across nine months	—
Oct 2025	₹13.71 cr	₹19.58 cr
Nov 2025	₹11.97 cr	₹16.92 cr
Dec 2025	₹7.46 cr	₹9.49 cr

The file contains **only October to December**. The naive parse reports ₹1.47 crore of revenue in January, ₹1.28 crore in March, and so on — for months that have no data at all. In total it moves **₹12.78 crore out of Q4 and into nine months the file does not cover**. A stakeholder looking at that chart sees a business with a weirdly strong first half and a Q4 spike, and every number is an artifact of one keyword argument.

(The correct-parse column is gross, before discounts, so it sits a little above the truth file's net monthly totals of ₹18.85, ₹16.29 and ₹9.11 crore — which is the expected direction. What matters here is that it contains **only** the three real months.)

Tier	What to say
Passes	"It is better, but I would check the remaining nulls"
Strong	Says no, and names the mechanism: <code>dayfirst</code> is applied column-wide, so it corrupts the ISO values while fixing the Indian ones
Extra points	+ [+Validate] the identical-NaT point: both parses report 49 nulls, so the bug is undetectable by any null-based check and only a value check finds it + [+Business] the month table above, because "your Q4 report shows revenue in March" is the version of this a stakeholder understands + [+Edge cases] the ambiguity is only visible on the 9,630 rows (37.1%) where both parts are ≤ 12 ; the other 13,072 rows have a day above 12 and parse correctly either way, which is why the bug survives spot-checking

Likely follow-ups: How would you detect this bug if you had not been told about it? What is the correct parse?

Red flag: believing a lower null count proves a better parse.

Learn it in: Chapter 14, §14.7; Chapter 25, §25.8.

Q72B-024 · So what is the correct way to parse this column?

Level: Mid · **Roles:** DA, DS, DE, AE

Remember it as: *Classify, then parse each class with an explicit format. One pass per shape, never one guess for the column.*

Answer in one line: Match each value's shape, then parse each group with its own **explicit** format string, handling Excel serials by date arithmetic rather than parsing — so nothing is inferred and anything unrecognised is left null on purpose, for review.

Measured, and it reconciles:

```
parsed = pd.Series(pd.NaT, index=s.index, dtype='datetime64[ns]')

m_dash = s.str.match(r'^\d{2}-\d{2}-\d{4}$')
m_slash = s.str.match(r'^\d{2}/\d{2}/\d{4}$')
m_iso = s.str.match(r'^\d{4}-\d{2}-\d{2}$')
m_ser = s.str.match(r'^\d{5}$')

parsed[m_dash] = pd.to_datetime(s[m_dash], format='%d-%m-%Y', errors='coerce')
parsed[m_slash] = pd.to_datetime(s[m_slash], format='%d/%m/%Y', errors='coerce')
parsed[m_iso] = pd.to_datetime(s[m_iso], format='%Y-%m-%d', errors='coerce')
parsed[m_ser] = (pd.Timestamp('1899-12-30')
                 + pd.to_timedelta(pd.to_numeric(s[m_ser]), unit='D'))

print('rows matched by a known shape:', (m_dash | m_slash | m_iso | m_ser).sum(), 'of', len(s))
print('still NaT', ':', parsed.isna().sum())
print('unparsed values', ':', sorted(s[parsed.isna()].unique()))
print('months present', ':', sorted(parsed.dropna().dt.to_period('M').astype(str).unique()))
```

```
rows matched by a known shape: 25969 of 25969
still NaT : 9
unparsed values : ['31-09-2025', '31-11-2025', '32-10-2025']
months present : ['2025-10', '2025-11', '2025-12']
```

Three results, each one a proof:

- **25,969 of 25,969** matched a known shape, so the parser is complete for this file.
- **9 nulls**, and they are the 9 genuinely impossible dates (Q72B-026) — not parsing casualties. Chapter 14's answer key records exactly 9.
- **Only the three real months appear.** The nine months of phantom revenue are gone.

Why `errors='coerce'` **is correct here and wrong in Q72B-022.** Same argument, opposite meaning: inside a branch that has already established the shape, a coerce failure means the value matched the shape but is not a real date — which is precisely the signal you want. Used on the whole column, it hides format mismatches instead.

Tier	What to say
Passes	<code>format='mixed', dayfirst=True</code> as a single call
Strong	The shape-dispatch approach with explicit formats per group, and the Excel serial handled by arithmetic from the 1899-12-30 epoch rather than by a format string
Extra points	+ [+Validate] the three proofs above — shapes sum to the row count, residual nulls are explained, and the month list contains only real months + [+Signpost] this is the same structure as Chapter 14's SQL solution, a CASE with one branch per shape, which is the honest way in every language + [+Trade-offs] it is more code than a one-liner, and that is the trade: five lines you can audit against one line that is wrong 34% of the time

Likely follow-ups: Why 1899-12-30 and not 1900-01-01? What do you do with the nine that are still null?

Red flag: no explicit format anywhere in the answer.

Learn it in: Chapter 14, §14.7 (and §14.13 for the SQL `CASE` version).

Q72B-025 · Why is the Excel date epoch 1899-12-30 rather than 1900-01-01?

Level: Mid · **Roles:** DA, AE, BA, DE

Remember it as: *Excel believes 1900 was a leap year. It was not. The epoch is shifted two days back to absorb the bug.*

Answer in one line: Because Excel's serial 1 is 1 January 1900 **and** Excel incorrectly treats 29 February 1900 as a real date — a deliberate compatibility bug inherited from Lotus 1-2-3 — so counting days from 1899-12-30 is what makes serials on or after 1 March 1900 come out right.

Measured:

```
print('serial 45933 ->', (pd.Timestamp('1899-12-30') + pd.Timedelta(days=45933)).date())
```

```
serial 45933 -> 2025-10-03
```

3 October 2025, which falls inside the file's quarter — the sanity check that confirms the epoch choice. Use 1900-01-01 and every one of the 40 serials lands two days late, which is small enough that nobody notices and large enough to move revenue between months at a month boundary.

Tier	What to say
Passes	Knows serials need converting and that the epoch is "around 1900"
Strong	Names the phantom 29 February 1900 and the Lotus compatibility reason, and why that makes 1899-12-30 the correct base for modern dates
Extra points	+ [+Edge cases] the Mac versions of Excel historically used a 1904 epoch, so a file from an older Mac is off by four years and a day + [+Validate] always sanity-check a converted serial against the period the data is supposed to cover, which is how you catch a wrong epoch immediately + [+Business] the presence of serials at all is the finding: someone opened the CSV in Excel and saved it, which will keep happening unless the handover process changes

Likely follow-ups: What does a serial with a decimal part mean? How would you convert these in SQL?

Red flag: `pd.to_datetime(45933, unit='D')`, which uses the Unix epoch and returns a date in 2095.

Learn it in: Chapter 14, §14.7; Chapter 11, §11.4 (how Excel stores dates).

Q72B-026 · Nine rows have the date 31-11-2025 . What do you do with them?

Level: Mid · **Roles:** DA, DS, DE, AE, BA

Remember it as: *An impossible date is not a parsing problem, it is a data problem. Null it, then repair it from a second source if one exists — and label the repair.*

Answer in one line: Leave them null rather than guessing, then look for an independent source for the real date — here `entered_at_utc`, which Riverstone's ERP team confirms is always the order date in Indian time — and repair from that, flagging every repaired row so downstream users can see which values were inferred.

The nine values, measured: 31-09-2025 , 31-11-2025 and 32-10-2025 . September and November have 30 days; no month has 32. These are data-entry errors, not format problems.

The repair, and why it is defensible. Chapter 14, §14.7 establishes the business rule: orders are always entered on the order date, in IST. So the entry timestamp is independent evidence. Note the format — `entered_at_utc` is ISO 8601 with a `z` suffix, so it needs `format='ISO8601'` , not a space-separated pattern:

```
ist = (pd.to_datetime(body['entered_at_utc'], format='ISO8601', utc=True)
      .dt.tz_convert('Asia/Kolkata'))

repaired = parsed.where(parsed.notna(), ist.dt.tz_localize(None).dt.normalize())
date_repaired = parsed.isna() & repaired.notna()

print('rows repaired:', date_repaired.sum())
print(pd.DataFrame({'id': body['order_item_id'][date_repaired],
                    'raw': s[date_repaired],
                    'entered_at_utc': body['entered_at_utc'][date_repaired],
                    'repaired to': repaired[date_repaired].dt.date}).to_string(index=False))
```

```
rows repaired: 9
   id      raw      entered_at_utc repaired to
192992 31-09-2025 2025-10-28T13:53:00Z 2025-10-28
193042 31-11-2025 2025-10-28T03:36:00Z 2025-10-28
193046 32-10-2025 2025-10-28T05:23:00Z 2025-10-28
193051 32-10-2025 2025-10-28T15:21:00Z 2025-10-28
201997 31-11-2025 2025-11-28T16:35:00Z 2025-11-28
202048 31-11-2025 2025-11-28T10:31:00Z 2025-11-28
202098 32-10-2025 2025-11-30T04:59:00Z 2025-11-30
209598 31-11-2025 2025-12-28T15:09:00Z 2025-12-28
209607 31-09-2025 2025-12-28T07:13:00Z 2025-12-28
```

Look at what the repair reveals: five of the nine land in a different month from the one that was written. Row 193042 says November and was entered on 28 October. Row 209607 says September and was entered on 28 December. Row 202098 says October and was entered on 30 November.

That is the strongest possible argument against the obvious shortcut. A cleaner who "fixed" 31-11-2025 to 30-11-2025 would have produced a value that looks entirely reasonable, passes every validation rule, and **assigns the row to the wrong month** — and in the case of row 209607, to the wrong quarter. The guess is undetectable; the evidence-based repair is checkable.

The `date_repaired` flag is the part that matters. Nine repaired rows out of 25,832 will never change a quarterly total, but the flag is what lets someone six months later ask "which dates did we infer?" and get an answer. A repair without a flag is indistinguishable from real data.

Tier	What to say
Passes	Nulls them and excludes them, or flags them for someone else to fix
Strong	Nulls them first, finds the independent source, repairs from it, and adds a flag column recording which rows were repaired and how
Extra points	+ [+Validate] the repair is checkable: the repaired date must equal the IST date of the entry timestamp, which is an assertion, not a hope + [+Edge cases] five of the nine rows move month under the repair, and one moves quarter, which proves why guessing "the 30th" would have been wrong + [+Business] nine rows is small, so the value here is not the correction but the discovery that the entry form accepts 31 November — a validation fix upstream prevents all future instances

Likely follow-ups: What if there were no entry timestamp? How would you detect an impossible date in SQL without the parse erroring?

Red flag: rounding 31-11-2025 to 30-11-2025 with no evidence and no flag.

Learn it in: Chapter 14, §14.7; Chapter 12, §12.9 (the month-survives trick in SQL).

Q72B-027 · The file has `order_date` and `entered_at_utc` . Why does that matter for a daily report?

Level: Mid · **Roles:** DA, DS, DE, AE, BA

Remember it as: *UTC and IST are five and a half hours apart, so for five and a half hours every day the two time zones disagree about what day it is.*

Answer in one line: Because `entered_at_utc` is in UTC and the business reports in IST (UTC+5:30), any timestamp from 18:30 UTC onward already belongs to the next Indian day — so grouping a daily report by the UTC date misfiles a predictable slice of every day's activity.

Measured, on this file:

```
utc = pd.to_datetime(body['entered_at_utc'], format='ISO8601', utc=True)
ist = utc.dt.tz_convert('Asia/Kolkata')

differs = utc.dt.date != ist.dt.date
print(f'rows where the UTC date is not the IST date: {differs.sum():,}')
print(f'as a share of the file: {differs.mean() * 100:.1f}%')
```

```
rows where the UTC date is not the IST date: 1,604
as a share of the file: 6.2%
```

6.2% is not a rounding error, and it is not random: it is every order entered between 18:30 and 23:59 IST. **The 5.5-hour offset is 23% of a day**, so the only reason the figure is 6.2% rather than higher is that most data entry happens during office hours. That also means the error is **systematically biased toward evening activity** — a report grouped on the UTC date will consistently understate evening orders and consistently pull them into the following day.

The half-hour matters too. India's offset is +5:30, not a whole number of hours, which breaks any code that stores offsets as integers. That is a real and common bug, and it is worth mentioning because it shows you have handled Indian time specifically rather than time zones in the abstract.

Tier	What to say
Passes	Knows the timestamp is UTC and should be converted
Strong	Converts explicitly to <code>Asia/Kolkata</code> , quantifies the affected rows, and explains that the error is a systematic evening bias rather than noise
Extra points	+ [+Edge cases] +5:30 is a half-hour offset, so an integer-hours representation cannot hold it + [+Validate] assert that the converted date equals <code>order_date</code> for the rows where <code>order_date</code> is trustworthy, which is both a time-zone check and a cross-check on the dates + [+Business] state the reporting convention in the output ("all dates are IST"), because the single most expensive version of this bug is two teams reporting the same day in two zones and each believing the other is wrong + [+Trade-offs] store UTC, display local: converting on the way in loses information that converting on the way out does not

Likely follow-ups: Would you store UTC or IST in the warehouse? What breaks in a country that observes daylight saving?

Red flag: `.dt.tz_localize('Asia/Kolkata')` on a UTC timestamp, which relabels it without converting and shifts every value by 5.5 hours.

Learn it in: Chapter 14, §14.7; Chapter 12, §12.10 (time zones in SQL).

Rapid-fire, 72B.4

Roles: DA, DS, DE, AE and BA for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q72B-028	What is the difference between <code>tz_localize</code> and <code>tz_convert</code> ?	<code>tz_localize</code> attaches a time zone to a naive timestamp without changing the clock reading; <code>tz_convert</code> changes the clock reading of an already-aware timestamp into another zone — so using the first where you need the second shifts every value by the offset	[+Edge cases] <code>tz_localize</code> on an already-aware timestamp raises, which is the one helpful error in this area	Mid · 14.7
Q72B-029	Why does comparing a tz-aware and a tz-naive timestamp fail?	pandas refuses because the comparison is genuinely undefined — a naive timestamp has no instant in time until you say which zone it is in — so it raises a <code>TypeError</code> rather than guessing	[+Trade-offs] the strictness is a feature: this is the one place the library refuses to guess, which is why the bug surfaces	Mid · 72.6
Q72B-030	A date column parses fine but every value is in 1970. What happened?	A Unix epoch timestamp was read as a date with the wrong unit — seconds read as nanoseconds, or a date arithmetic that defaulted to the 1970 epoch — so the values cluster at or just after 1970-01-01	[+Validate] a min date of 1970-01-01 is the single most recognisable parsing-failure signature	Fresher · 14.7
Q72B-031	How do you detect an impossible date in SQL without the query erroring?	Build the first of the month and add the day offset, then check whether the month is still the one written: <code>31-11-2025</code> becomes 1 December, so the month no longer matches and the row is flagged without <code>to_date</code> ever raising	[+Signpost] this is the technique Chapter 14 uses on all 25,832 lines	Mid · 14.13, 12.9
Q72B-032	Your date column is 37% ambiguous. How do you decide day-first or month-first?	Not by guessing: use the rows that resolve themselves — a value with a first part above 12 can only be day-first — and if a file contains both conventions, find a column that disambiguates, such as an entry timestamp, or ask the source	[+Validate] on this file 13,072 rows have a day above 12, which settles the convention for the <code>dd-mm</code> block with evidence	Senior · 14.7
Q72B-033	Why store dates as <code>DATE</code> rather than text in a warehouse?	Because text sorts lexically rather than chronologically (so <code>02-01-2026</code> sorts before <code>31-12-2025</code>), accepts impossible values such as <code>31-11-2025</code> , cannot do date arithmetic, and defeats every partition and range optimisation the database has	[+Business] the type is the cheapest validation rule you will ever get, and it is enforced on every future insert	Fresher · 12.4, 14.13

72B.5 Categories and text: eighteen spellings of four things

Q72B-034 · `status` has 18 distinct values and should have 4. How do you fix it, and what does lowercasing alone get you?

Level: Fresher · **Roles:** DA, DS, DE, AE, BA

Remember it as: *Trim and lowercase is the first 60% of the job. The rest needs a mapping table, because no function knows that `Dlvd` means `Delivered`.*

Answer in one line: Normalise first — trim whitespace, collapse internal spaces, lowercase — which collapses 18 spellings to 7, then apply an explicit **mapping table** for the abbreviations and genuine variants that normalisation cannot reach, and assert that nothing is left unmapped.

Measured, the whole funnel:

```
print('raw distinct      :', body['status'].nunique())
print(sorted(body['status'].unique()))

norm = body['status'].str.strip().str.lower()
print('after trim + lower :', norm.nunique(), sorted(norm.unique()))
```

```
raw distinct      : 18
['CANCELLED', 'Canceled', 'Cancelled', 'Cxl', 'DELIVERED', 'Delivered', 'Delivered ',
 'Dlvd', 'PENDING', 'Pending', 'Pending ', 'SHIPPED', 'Shipped', 'Shipped ',
 'cancelled', 'delivered', 'pending', 'shipped']
after trim + lower : 7 ['canceled', 'cancelled', 'cxl', 'delivered', 'dlvd', 'pending', 'shipped']
```

18 → 7, and then it stops. The remaining three problems are not formatting:

- `canceled` and `cancelled` — American and British spelling, both real words
- `cxl` — a trade abbreviation
- `dlvd` — another one

No string function will ever resolve those, because resolving them requires knowing the business. That is what the mapping table is for, and Chapter 14 ships it as `status_map.csv`:

```
status_map = {'delivered': 'Delivered', 'dlvd': 'Delivered',
              'canceled': 'Cancelled', 'cancelled': 'Cancelled', 'cxl': 'Cancelled',
              'shipped': 'Shipped', 'pending': 'Pending'}

clean_status = norm.map(status_map)
print('unmapped:', clean_status.isna().sum())
print(clean_status.value_counts().to_string())
```

```
unmapped: 0
Delivered    22551
Pending       1174
Shipped       1144
Cancelled    1100
```

The map needs exactly seven entries, one per normalised value, and `cxl` is the one that gets forgotten. It is worth knowing what happens when it is: leave `cxl` out and the same code reports `unmapped: 32` with `Cancelled` at 1,068 instead of 1,100. Thirty-two cancelled lines become null, drop out of every `groupby`, and quietly raise the delivered share. The assertion in Q72B-035 is what turns that from a silent 32-row error into a one-line failure message — and this is not a hypothetical, it is what happened the first time these counts were produced for this chapter.

(These counts are the 25,969-row body, which still contains the 137 duplicates. After de-duplication they become 22,434 / 1,166 / 1,138 / 1,094, which is what Chapter 14's truth file holds. Both are correct for their stage; the point of Q72B-015 is that you always know which stage you are quoting.)

Why the naive version is so convincing and so wrong. Group by the raw column and `Delivered` comes back as 19,835 — which looks like a perfectly plausible number:

Delivered	19835
Pending	1026
Shipped	1004
Cancelled	985
DELIVERED	697
Dlvd	694

Nothing about 19835 announces that 2,716 delivered lines are sitting further down the list under five other spellings. A report built on it understates deliveries by 12% and nobody can tell from the output.

Tier	What to say
Passes	<code>.str.lower().str.strip()</code> and considers the column fixed
Strong	The normalise-then-map funnel, with the explicit point that normalisation gets 18 to 7 and only a business mapping closes the last three
Extra points	+ [+Validate] <code>assert clean_status.notna().all()</code> so a new spelling in next month's file fails the pipeline loudly instead of becoming a silent null + [+Business] a mapping table belongs in a reviewable file, not inline in code, so the business owner of the definition can read and change it without touching Python + [+Edge cases] <code>Delivered</code> with a trailing space is invisible in every preview, in Excel and in a printed table, which is why trimming comes before looking

Likely follow-ups: Where should the mapping table live, and who owns it? What happens when next month's file has a nineteenth spelling?

Red flag: a chain of `.replace()` calls hard-coded in the script, one per spelling found.

Learn it in: Chapter 14, §14.8 (mapping tables); Chapter 25, §25.10.

Q72B-035 · What should happen when next month's file contains a spelling your mapping table has never seen?

Level: Mid · **Roles:** DA, DE, AE

Remember it as: *Fail loudly. A mapping that silently passes unknown values through is how a nineteenth spelling becomes a nineteenth category in the dashboard.*

Answer in one line: The pipeline should stop, or quarantine the affected rows and alert — never silently null them, and never pass the raw value through — because an unmapped value is new information about the source, which is exactly the thing you want to hear about.

The three designs, and why only one is safe:

Design	What happens to Dispatched	Verdict
<code>.map(status_map)</code> with no check	Becomes NaN, drops out of every <code>groupby</code> , revenue silently disappears	Dangerous
<code>.replace(status_map)</code>	Passes through unchanged, appears as a 5th category in the dashboard	Dangerous, differently
<code>.map()</code> + assert, or map with quarantine	Pipeline fails, or the rows are set aside and reported	Correct

The middle one deserves a word, because it looks safer and is often worse. `.map()` nulls unknown values; `.replace()` leaves them. With `.replace()` the new value propagates into the report looking like a legitimate category, and the first person to notice is a stakeholder asking what "Dispatched" means — by which point the number has been circulating for a month.

```
assert clean_status.notna().all(), \
    f'unmapped status values: {sorted(set(norm[clean_status.isna()])))}'
```

One line, and it turns a silent corruption into a named failure with the offending values printed.

Tier	What to say
Passes	"I would add it to the mapping table" — true, but says nothing about how you find out
Strong	Names the fail-loudly principle, distinguishes <code>.map()</code> from <code>.replace()</code> by their unknown-value behaviour, and shows the assertion
Extra points	+ [+Validate] the assertion message lists the unmapped values, so the fix takes seconds rather than an investigation + [+Business] a new status spelling usually means a real process change upstream — a new warehouse, a new ERP release, a new team — so this alert is a business signal, not just a data error + [+Trade-offs] quarantine-and-continue beats hard-fail when the report must ship daily and a few unmapped rows are tolerable, as long as the quarantine count is visible on the output

Likely follow-ups: Would you fail the whole run or just those rows? How do you stop the mapping table becoming a thousand-row mess?

Red flag: `.replace()` with no check, or `.fillna('Other')`, which buries the signal in a bucket.

Learn it in: Chapter 14, §14.8 and §14.12; Chapter 45, §45.8 (schema drift).

Q72B-036 · branch has 17 spellings for 4 branches. Normalisation gets you to 12. What are the other 8, and why is that question harder than it looks?

Level: Mid · **Roles:** DA, AE, BA

Remember it as: *Abbreviations, old city names and organisational suffixes. Every one of them needs a person who knows the business — and one of them is a judgement call.*

Answer in one line: The residue is abbreviations (BLR, DEL, KOL), historical city names (Bangalore, Calcutta), punctuation variants (Mumbai H.O.), a bare form missing the suffix (Mumbai), and New Delhi — which is genuinely ambiguous and must be settled by a business rule rather than by a cleaner's assumption.

Measured:

```
print('raw distinct      :', body['branch'].nunique())
print(sorted(body['branch'].unique()))
norm_b = body['branch'].str.strip().str.lower()
print('after trim + lower:', norm_b.nunique(), sorted(norm_b.unique()))
```

```
raw distinct      : 17
['BLR', 'Bangalore', 'Bengaluru', 'Calcutta', 'DEL', 'Delhi', 'KOL', 'Kolkata',
 'MUMBAI HO', 'Mumbai', 'Mumbai H.O.', 'Mumbai HO', 'New Delhi', 'bengaluru',
 'delhi', 'kolkata', 'mumbai ho']
after trim + lower: 12 ['bangalore', 'bengaluru', 'blr', 'calcutta', 'del', 'delhi',
                        'kol', 'kolkata', 'mumbai', 'mumbai h.o.', 'mumbai ho', 'new delhi']
```

The full 12-row map resolves to four branches with nothing unmapped. On the 25,969-row body:

```
Mumbai HO    9223
Bengaluru    7218
Delhi        6301
Kolkata      3227
```

and after the 137 duplicates are removed those become **9,163 / 7,185 / 6,274 / 3,210**, which match Chapter 14's answer key exactly. The two sets of numbers differ by precisely the duplicate lines — which is itself a useful check, and a reminder to state which stage of the pipeline any figure comes from.

The hard part is `New Delhi`. New Delhi is a district *within* Delhi, so it is not a misspelling and not an abbreviation — a person has to decide whether it is the same branch. Riverstone's sales regions treat it as Delhi, so the map follows that **stated business rule**, and the rule is recorded alongside the mapping. The same applies to `Mumbai` without the `HO` suffix: that is an inference that the one Mumbai branch is the head office, which happens to be true here and would not be in a company with two Mumbai sites.

This is the interview point. A candidate who maps `New Delhi` → `Delhi` without comment has guessed. A candidate who says "New Delhi is ambiguous; I would confirm with the business whether it is a separate branch, and record the answer" has demonstrated the judgement the question is actually testing. Chapter 14 makes the same point about city names: no function turns **Bombay** into **Mumbai** or **Vizag** into **Visakhapatnam**, because those mappings live in a person's knowledge of India, not in a library.

Tier	What to say
Passes	Lists the obvious variants and maps them
Strong	Categorises the residue — abbreviations, historical names, punctuation, missing suffix — and flags <code>New Delhi</code> as a decision rather than a cleaning step
Extra points	+ [+Clarify] ask the business about <code>New Delhi</code> and about whether <code>Mumbai</code> could be a second site, rather than assuming + [+Validate] the mapped line counts reconcile to the answer key, and the mapped total equals the input total so no row was lost in the mapping + [+Business] the abbreviations tell you which offices use a code and which type a name, which is a data-entry standardisation worth fixing at the form

Likely follow-ups: Where does the mapping table live so the business can own it? How would you handle a genuinely new branch opening mid-quarter?

Red flag: mapping `New Delhi` → `Delhi` silently, or treating it as obviously a typo.

Learn it in: Chapter 14, §14.8 (and §14.15 for the city rules).

Q72B-037 · 880 rows have a trailing space in `sales_rep`, turning 12 reps into 23. Why is whitespace the most dangerous kind of dirt?

Level: Fresher · **Roles:** DA, DS, DE, AE, BA

Remember it as: *Whitespace is dirt you cannot see. Every other data-quality problem at least looks wrong on screen.*

Answer in one line: Because it is invisible in every tool a human uses to inspect data — previews, Excel cells, printed tables, dashboards — so `"Irfan Sheikh "` and `"Irfan Sheikh"` look identical while grouping as two separate people, and the error is never caught by eye.

Measured:

```
print('rows with a trailing or leading space:', (body['sales_rep'] !=
body['sales_rep'].str.strip()).sum())
print('distinct reps, raw      :', body['sales_rep'].nunique())
print('distinct reps, stripped:', body['sales_rep'].str.strip().nunique())
```

```
rows with a trailing or leading space: 880
distinct reps, raw      : 23
distinct reps, stripped: 12
```

Twelve sales reps became twenty-three. On a leaderboard, every affected rep appears twice with their numbers split between the two entries, and both entries look like plausible reps with plausible figures. There is no visual cue at all.

The fix is trivial and the discipline is the point: **strip every text column on load**, as a blanket rule, before you look at anything. It costs nothing and removes an entire category of invisible error.

```
for c in body.select_dtypes('object').columns:
    body[c] = body[c].str.strip()
```

Tier	What to say
Passes	<code>.str.strip()</code> on the affected column
Strong	The invisibility argument, the measured 12-becomes-23 consequence, and stripping as a blanket rule applied on load rather than per column when a problem is noticed
Extra points	+ [+Edge cases] trailing spaces are only the common case: non-breaking spaces (<code>\xa0</code> , typical of data pasted from a web page or Word), tabs, and zero-width characters all survive <code>.strip()</code> in some tools, so <code>.str.replace(r'\s+', ' ', regex=True)</code> plus an explicit <code>\xa0</code> replacement is the thorough version + [+Validate] assert that no value differs from its stripped form after cleaning, which is a one-line permanent guard + [+Business] a split leaderboard is the version of this that gets noticed, usually by the rep whose numbers halved, which is an expensive way to find a bug

Likely follow-ups: Does `TRIM` in SQL remove a non-breaking space? How would you find a zero-width character?

Red flag: not stripping text columns by default.

Learn it in: Chapter 14, §14.8; Chapter 12, §12.6 (`TRIM` in SQL).

Q72B-038 · The CRM has 12 spellings of `segment` for 3 segments, and 64 city values for 39 cities. How do you approach a column where you do not know the right answer?

Level: Mid · **Roles:** DA, AE, BA

Remember it as: *Build the candidate mapping, then get it signed off. You are proposing definitions, not discovering them.*

Answer in one line: Normalise, group the remaining variants into proposed clusters, and take the proposed mapping to whoever owns the definition for confirmation — because deciding that `HoReCa` and `Hotel/Restaurant` are one segment is a business decision you are not authorised to make alone.

Measured, the segment column:

```
print(cust['segment'].nunique(), sorted(cust['segment'].unique()))

seg = (cust['segment'].str.strip().str.lower()
       .replace({'horeca': 'hospitality', 'hotel/restaurant': 'hospitality',
                 'distributor': 'wholesale'}))
print('after mapping:', seg.nunique(), dict(seg.value_counts()))
```

```
12 ['Distributor', 'HoReCa', 'Hospitality', 'Hotel/Restaurant', 'RETAIL', 'Retail',
    'Retail ', 'WHOLESALE', 'Wholesale', 'hospitality', 'retail', 'wholesale']
after mapping: 3 {'retail': 2791, 'hospitality': 1513, 'wholesale': 723}
```

Three of those twelve are substantive, and each is a decision:

- `HoReCa` is an industry term for Hotel/Restaurant/Café. Mapping it to Hospitality is near-certain but still a mapping someone should confirm.

- Hotel/Restaurant likewise.
- Distributor → Wholesale is the genuinely debatable one. A distributor and a wholesaler are not always the same commercial relationship, and if they are priced differently or reported separately, merging them destroys a distinction the business cares about.

Chapter 14, §14.15 records the rule Riverstone uses, which is the right outcome: **the decision exists, it is written down, and the cleaning script implements it rather than inventing it.**

Tier	What to say
Passes	Normalises and merges the variants that look the same
Strong	Separates the mechanical variants from the three substantive decisions, and takes the substantive ones to the definition owner before implementing
Extra points	+ [+Clarify] Distributor → Wholesale is the one to question explicitly, because it may be a real commercial distinction rather than a spelling + [+Validate] the mapped counts must sum to the record count, so no customer silently falls out of segmentation + [+Business] present the proposal as a table of "raw value → proposed segment → row count", because a business owner can review that in two minutes and cannot review a Python dictionary + [+Edge cases] city needs India-specific knowledge no library has — Bombay, Madras, Poona, Vizag, Trivandrum, Mysore — and New Delhi is the same judgement call as Q72B-036

Likely follow-ups: What if the business owner disagrees with your clustering? How would you keep the mapping current as new values appear?

Red flag: merging Distributor into Wholesale without flagging it as a decision.

Learn it in: Chapter 14, §14.8 and §14.15.

Rapid-fire, 72B.5

Roles: DA, DS, DE, AE and BA for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q72B-039	<code>.map()</code> or <code>.replace()</code> for a category mapping?	<code>.map()</code> , because it turns unknown values into <code>NaN</code> where an assertion can catch them, whereas <code>.replace()</code> passes unknown values through silently so a new variant reaches the report looking legitimate	[+Validate] <code>.map()</code> plus one assert is the whole pattern	Mid · 14.8
Q72B-040	Why keep mapping tables in CSV files rather than in code?	Because the business owns the definitions, not the engineer: a CSV can be reviewed, versioned, diffed and edited by the person who knows what <code>Distributor</code> means, without a code change or a deploy	[+Business] it also makes the mapping auditable, which matters when a number is challenged	Mid · 14.8
Q72B-041	When is fuzzy matching the right tool for categories?	Rarely, and only after deterministic normalisation and an explicit map have taken the obvious cases — for the long tail of genuine typos in free-text entry, with every proposed match reviewed by a person before it is applied	[+Trade-offs] any threshold trades false merges against missed matches, and which error is worse is a business question	Senior · 14.6
Q72B-042	<code>.str.lower()</code> versus <code>.str.casefold()</code> ?	<code>casefold</code> is more aggressive and handles non-English cases correctly (German <code>ß</code> folds to <code>ss</code>), so it is the better default for matching text across languages; for ASCII business categories the two are identical	[+Edge cases] Turkish dotless <code>i</code> is the classic case where naive lowercasing breaks a match	Mid · 17.8
Q72B-043	A category column has <code>'', 'N/A', '-'</code> , <code>'unknown'</code> and <code>'NULL'</code> . Same thing?	Mechanically they all mean "no value", but they are not the same <i>finding</i> : a blank usually means the field was skipped while a typed <code>N/A</code> means someone deliberately recorded that it does not apply, and conflating them loses that distinction	[+Business] on this book's CRM file 100 city values are missing across these forms, and knowing which were deliberate changes the upstream fix	Mid · 14.5
Q72B-044	Why is <code>category</code> dtype worth using after cleaning?	Because a cleaned categorical column has few distinct values and many rows, so pandas stores small integer codes plus one copy of each label instead of a full string per row — a large memory saving, and <code>groupby</code> gets faster	[+Edge cases] a <code>category</code> column silently produces <code>NaN</code> if you assign a value outside its categories, which is a feature here: it is the same fail-loudly guard as <code>.map()</code>	Mid · 25.11

72B.6 Numbers, units and money

Q72B-045 · 2,316 prices are written as Rs. 430 or ₹430.00. What does `pd.to_numeric(col, errors='coerce')` cost you?

Level: Mid · **Roles:** DA, DS, DE, AE, BA

Remember it as: `errors='coerce'` on a money column does not skip 2,316 rows. It silently deletes ₹2.02 crore.

Answer in one line: It turns all 2,316 currency-text values into NaN, and because `.sum()` skips nulls by default, the revenue total comes out **₹2.02 crore (4.6%) lower** with no error, no warning and no missing-row count anywhere in the output.

Measured:

```
qty    = pd.to_numeric(body['quantity'], errors='coerce')
p_lazy = pd.to_numeric(body['unit_price'], errors='coerce')

print(f'prices turned to NaN : {p_lazy.isna().sum():,}')
print(f'gross with them null : {(qty * p_lazy).sum():>15,.0f}')
```

```
prices turned to NaN : 2,316
gross with them null :    418,690,095
```

The true gross is **₹46.01 crore**. That figure is **₹41.87 crore** — `errors='coerce'` plus pandas' null-skipping `.sum()` quietly removed **₹4.14 crore, or 9.0% of revenue**, with no error and no warning. Either default alone would be survivable; together they turn a type problem into a silent financial misstatement.

Now the trap in the obvious fix — and this one is worth the price of the chapter. The natural cleaner is "strip everything that is not a digit or a dot":

```
naive = pd.to_numeric(body['unit_price'].str.replace(r'^\0-9.', '', regex=True),
                      errors='coerce')
print('distinct prices:', sorted(naive.dropna().unique()))
```

```
distinct prices: [0.115, 0.14, 0.29, 0.38, 0.43, 0.62, 0.75,
                  115.0, 290.0, 380.0, 430.0, 620.0, 750.0, 1400.0]
```

Fourteen prices, half of them nonsense. `Rs. 1,400` became **0.14**. The regex deleted `R`, `s` and the space but kept the **dot from "Rs."**, leaving `.1400`. The 1,205 `Rs.`-prefixed rows are destroyed; the 1,111 rupee-sign rows (`₹1,400.00` → `1400.00`) come through fine, which is exactly why the bug survives a spot check.

'Rs. 1,400'	naive strip → 0.14 · correct → 1400
'Rs. 430'	naive strip → 0.43 · correct → 430
'₹1,400.00'	naive strip → 1400.0 · correct → 1400.0

A total built on it reads ₹43.89 crore — **₹2.13 crore (4.6%) short**, and far more plausible than the ₹41.87 crore the lazy version gives. The better-looking fix produces the more believable wrong answer.

The cleaner that works, and the check that proves it. Remove the known currency tokens rather than keeping a character class:

```
p_clean = pd.to_numeric(body['unit_price']
                        .str.replace(r'(?i)^rs\.\s*', '', regex=True)
                        .str.replace('₹', '', regex=False)
                        .str.replace(',', '', regex=False), errors='coerce')

assert p_clean.notna().all(), f'{p_clean.isna().sum()} prices still unparseable'
print('distinct prices:', sorted(p_clean.unique()))
print(f'gross: {(qty * p_clean).sum():>15,.0f}')
```

```
distinct prices: [115.0, 290.0, 380.0, 430.0, 620.0, 750.0, 1400.0]
gross: 460,139,670
```

Twenty-one spellings become **seven prices, zero nulls**. The three totals are worth putting side by side, because all three are produced by code that looks reasonable:

Approach	Gross	Error
<code>to_numeric(errors='coerce')</code>	₹41.87 cr	−₹4.14 cr (9.0%)
<code>Strip [^0-9.]</code>	₹43.89 cr	−₹2.13 cr (4.6%)
Remove known tokens	₹46.01 cr	correct

Tier	What to say
Passes	"Those rows would become null" — correct, with no sense that the total changes
Strong	Quantifies the loss and names the two interacting defaults: <code>coerce</code> creates nulls, <code>.sum()</code> skips them, so the error is invisible in the output
Extra points	+ [+Validate] assert zero nulls after a money parse, always, because on a financial column a null is never acceptable and the assertion costs one line + [+Edge cases] a character-class strip also breaks on a negative written as (430) or with a Unicode minus, and on European 1.400,00 where the comma is the decimal separator — remove the tokens you profiled rather than keeping a class of characters, and assert the result against the known set of values + [+Business] both error sizes — 9.0% and 4.6% — are larger than any rounding tolerance and smaller than an obvious error, which is the worst possible size: big enough to matter, small enough to be believed + [+Validate] the strongest check here is set membership, not nullness: seven known prices means <code>assert p_clean.isin(KNOWN).all()</code> , which catches 0.14 instantly where a null check cannot

Likely follow-ups: How would you handle a column with both 1,400.00 and 1.400,00? What if some prices were negative?

Red flag: reaching for `errors='coerce'` on a money column without a null check afterwards.

Learn it in: Chapter 14, §14.9; Chapter 25, §25.7.

Q72B-046 • `discount_pct` holds both 5 and 0.05. How do you tell which is which, and what does getting it wrong cost?

Level: Senior • **Roles:** DA, DS, AE, BA

Remember it as: *The same column records percent and fraction. No type check can tell them apart, because both are valid numbers.*

Answer in one line: You cannot tell from the type — both parse cleanly — so you infer from the value range given the business rule that a discount below 1% is implausible, convert the 1,371 fractional rows by multiplying by 100, and **confirm the rule with the business** rather than assuming it.

Measured:

```
print('distinct values:', sorted(body['discount_pct'].unique()))

d = pd.to_numeric(body['discount_pct'], errors='coerce')
print('rows strictly between 0 and 1:', ((d > 0) & (d < 1)).sum())

d_fixed = d.where(d > 1, d * 100)

r_lazy = (qty * p_clean * (1 - d / 100)).sum()
r_fixed = (qty * p_clean * (1 - d_fixed / 100)).sum()
print(f'net, discount as-is : {r_lazy:>15,.0f}')
print(f'net, fractions fixed : {r_fixed:>15,.0f}')
print(f'overstated by : {r_lazy - r_fixed:>15,.0f} ({(r_lazy / r_fixed - 1) * 100:.2f}%)')
```

```
distinct values: ['0', '0.05', '0.08', '0.10', '0.12', '10', '12', '5', '8']
rows strictly between 0 and 1: 1371
net, discount as-is      :    445,753,198
net, fractions fixed    :    443,796,562
overstated by           :         1,956,636 (0.44%)
```

Nine distinct values, and they pair up exactly: $0.05/5$, $0.08/8$, $0.10/10$, $0.12/12$. **That pairing is the evidence**, and it is far stronger than the range heuristic alone — four fractions each with a matching percent is not a coincidence, it is two offices recording the same four discount tiers in two conventions.

Why this is the subtlest error in the chapter. ₹19.6 lakh on ₹44.4 crore is 0.44%. It will never trip a sanity check, never look wrong on a chart, and never be questioned. It is also the error most likely to survive into production, because unlike the price column there is nothing malformed to notice: every value is a clean number.

The honest caveat. `d.where(d > 1, d * 100)` treats any value below 1 as a fraction, which would misread a genuine 0.5% discount as 50%. On this file no such discount exists, and the paired tiers confirm it — but the rule is an inference about *this* data and must be stated as one, not applied blindly to the next file.

Tier	What to say
Passes	Spots that two conventions are mixed and multiplies the small ones by 100
Strong	Uses the paired-tier evidence rather than a bare range heuristic, quantifies the error, and states the inference's limit — a real 0.5% discount would break the rule
Extra points	+ [+Clarify] confirm the discount tiers with the business, which converts an inference into a documented rule + [+Validate] after conversion, assert every discount is one of the four known tiers, which is a far tighter check than a range + [+Business] the finding to report upstream is that two offices use two conventions in one field, which is a form-design fix + [+Edge cases] a value of exactly 1 is genuinely ambiguous — 1% or 100%? — and on this file none exists, which is worth confirming rather than assuming

Likely follow-ups: What if a real 0.5% discount existed? How would you store this column to prevent the problem recurring?

Red flag: applying the fraction rule with no stated assumption and no check on the resulting values.

Learn it in: Chapter 14, §14.9; Chapter 10, §10.6 (defining a metric precisely).

Q72B-047 · 141 rows have `qty_unit = 'CTN'` and the rest `'PCS'`. What breaks if you ignore it?

Level: Mid · **Roles:** DA, DS, AE, BA

Remember it as: *Summing a quantity column with two units in it produces a number with no unit at all.*

Answer in one line: Summing `quantity` across both units adds cartons to pieces and yields a meaningless total — and because cartons hold more than one piece, the result **understates** true volume, while every per-unit metric built from it (units per order, price per unit, stock cover) is also wrong.

Measured:

```
print(dict(body['qty_unit'].value_counts()))

ctn = body['qty_unit'] == 'CTN'
print(f'carton rows      : {ctn.sum()}')
print(f'their quantity sum : {qty[ctn].sum():.0f}')
```

```
{'PCS': 25828, 'CTN': 141}
carton rows      : 141
their quantity sum : 540
```

The conversion factor is not in the file, and that is the whole question. Chapter 14's truth file settles it — those same 141 rows sum to **5,400** once cleaned, so Riverstone's factor is **1 carton = 10 pieces**, and the raw figure understates those rows by 4,860 units.

Worth being blunt about how this section was written: the first draft assumed twelve per carton.

Twelve is the intuitive answer, it is the common retail case, and it is wrong here. Nothing in the order file would ever have revealed that — only the product master does. The error would have been 20% on every carton row, with no symptom.

That is the answer to the question. The factor comes from the product master or the business, never from intuition, and it may vary by product — a carton of 1,400-rupee items need not hold the same count as a carton of 115-rupee items. On a 25,969-row file the effect on the total is small, but **on those customers' records it is wrong by 900%**, and anyone analysing one customer or one product sees nonsense.

Tier	What to say
Passes	Spots the second unit and converts it
Strong	Converts, and flags that the factor must come from the product master rather than an assumption, and that it may vary by product
Extra points	+ [+Validate] after conversion, assert that <code>qty_unit</code> has exactly one distinct value, which makes the column's unit a provable property rather than a hope + [+Clarify] state the factor and its source in the answer — "10 per carton, from the product master" — because an unsourced factor is a guess however confident it sounds + [+Business] a mixed-unit quantity column is a schema problem: the clean design is to store a canonical unit always, with the entry form doing the conversion + [+Edge cases] the carton rows are only 0.5% of the file, so they will never show up in a total-level sanity check and will look obviously wrong in any per-customer view

Likely follow-ups: What if the carton size varied by product? How would you validate your assumed factor?

Red flag: `qty.sum()` across both units, or inventing a conversion factor — twelve, say — without naming it as an assumption and sourcing it.

Learn it in: Chapter 14, §14.9; Chapter 28, §28.6 (units in a fact table).

Q72B-048 · Six rows have a quantity of 700 where the median is 35. Error or reality?

Level: Mid · **Roles:** DA, DS, AE, BA

Remember it as: *An outlier is a question, not a verdict. Answer it with a second source, not with a percentile.*

Answer in one line: Almost certainly data-entry errors — a typed extra zero — and the way to establish that is to compare against the **historical maximum** rather than a statistical cutoff, because the question "is 700 possible?" is answered by the business's past, not by this file's distribution.

Measured:

```
print(f'median {qty.median():.0f} | p95 {qty.quantile(.95):.0f} '
      f'| p99 {qty.quantile(.99):.0f} | max {qty.max():.0f}')

hist_max = 90          # highest quantity on any order line before Q4 2025
print('rows above the historical max:', (qty > hist_max).sum())
```

median 35 | p95 70 | p99 85 | max 700
rows above the historical max: 6

The historical-maximum rule catches exactly six rows, and they are exactly the six planted typos.

That is a clean result and it is worth dwelling on why it worked where a statistical rule would not:

- **p99 is 85.** A "drop anything above p99" rule would discard about 260 legitimate rows and still keep nothing useful, because the six bad rows are so extreme they barely move the percentile.
- **An IQR or 3-sigma rule** is distorted by the very outliers it is meant to find, and has no way to know that 90 is the real ceiling.
- **The historical maximum is evidence.** Someone checked what the business has actually ever shipped on one line. That is a fact about the world, which is what you need.

And the truth file confirms the diagnosis precisely:

Raw quantity	700	400	450	350	450	100
True quantity	70	40	45	35	45	10

Every one is the real value with one extra zero, inflating total volume by **2,205 units**. Note the last pair: `100` → `10`. A quantity of 100 looks entirely unremarkable next to a median of 35, and only the historical-maximum rule — not a glance, not a percentile — flags it.

What to actually do with them. Quarantine, not silent correction. Dividing by ten is a confident guess; it is probably right and it is still a guess. The right action is to set the six rows aside, report them, and let the business confirm — which is what Chapter 14's `dq_order_issues_pandas.csv` quarantine file is for.

Tier	What to say
Passes	Flags them as outliers and suggests removing or capping them
Strong	Treats it as a question, uses the historical maximum as external evidence, and quarantines rather than correcting
Extra points	+ [+Validate] the rule catches exactly 6 rows, which is itself evidence the threshold is right — a rule that flagged 300 rows would be the wrong rule + [+Edge cases] the <code>100</code> → <code>10</code> row is the one that proves the method, because it is invisible to any eyeball check + [+Business] quarantine and ask; a silent divide-by-ten is indistinguishable from real data six months later + [+Trade-offs] a hard threshold needs maintaining as the business grows, so it belongs in a config file with a date and a source, not hard-coded

Likely follow-ups: What if one of the six turned out to be a genuine bulk order? How would you set the threshold if there were no history?

Red flag: capping at p99, or deleting outliers because they are outliers.

Learn it in: Chapter 14, §14.6 and §14.12; Chapter 36, §36.4 (outliers that are real).

Rapid-fire, 72B.6

Roles: DA, DS, DE, AE and BA for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q72B-049	Why never store money as a float?	Because binary floating point cannot represent most decimal fractions exactly, so repeated addition accumulates error and two mathematically equal totals can compare as unequal; use <code>DECIMAL</code> / <code>NUMERIC</code> in the database and <code>Decimal</code> or integer paise in code	[+Edge cases] <code>0.1 + 0.2 != 0.3</code> is the one-line demonstration, and it is a reconciliation failure waiting to happen	Mid · 12.4, 17.6
Q72B-050	What is wrong with <code>.fillna(0)</code> on a price column?	It silently converts "we do not know the price" into "the price was zero", which is a different claim entirely and pulls every average down while leaving the row count unchanged, so nothing looks missing	[+Business] zero is a value, not an absence; the distinction is the whole point of null	Fresher · 14.5
Q72B-051	<code>.sum()</code> skips nulls but <code>.sum()</code> of an all-null column returns what?	<code>0.0</code> , not null — so a column that failed to parse entirely produces a confident zero total rather than an error, which is the most dangerous possible failure mode for a revenue figure	[+Validate] check the non-null count alongside every sum	Mid · 25.6
Q72B-052	How do you handle a column with two currencies in it?	Never convert silently: store the amount and its currency code as separate columns, and convert only at the point of reporting, with the rate's date recorded — because a converted figure without a rate and a date is not reproducible	[+Business] the rate date is part of the number; without it the total cannot be recomputed	Mid · 14.9
Q72B-053	A percentage column contains <code>15%</code> as text. Fix?	Strip the <code>%</code> and divide by 100 if you want a fraction, or strip and keep the number if you want percent — and whichever you choose, put the convention in the column name (<code>discount_pct</code> vs <code>discount_frac</code>) so the next person cannot get it wrong	[+Signpost] this is Q72B-046's problem prevented by naming	Fresher · 14.9
Q72B-054	Why is <code>int64</code> the wrong dtype for a column with blanks?	Because NumPy's <code>int64</code> has no null representation, so pandas promotes the column to <code>float64</code> and every ID prints as <code>101.0</code> ; pandas' nullable <code>Int64</code> holds integers and <code>NA</code> together, which is what an ID column with gaps needs	[+Edge cases] this is why <code>product_id</code> came back as a float in Q72B-002	Mid · 25.5
Q72B-055	How do you round a half?	State the rule: Python's <code>round()</code> uses banker's rounding (<code>round(0.5)</code> is <code>0</code> , <code>round(1.5)</code> is <code>2</code>), while most business conventions expect half-up — so a finance total must specify which, and <code>Decimal</code> with an explicit <code>ROUND_HALF_UP</code> is how you get it	[+Business] an unstated rounding rule is a recurring reconciliation dispute	Mid · 17.6
Q72B-056	Should you round before or after aggregating?	After: rounding each row first and then summing accumulates the per-row error, so sum at full precision and round the presented figure once	[+Validate] rounding before aggregation is a classic cause of a total that does not match its own parts	Mid · 10.6

72B.7 Keys, joins and match rates

Q72B-057 · Your order file has `customer_code` as 124 and the master has 0124 . What happens, and what does it cost?

Level: Mid · **Roles:** DA, DS, DE, AE, BA

Remember it as: *A broken join key does not raise an error. It returns fewer rows, and an inner join makes the loss invisible.*

Answer in one line: The codes do not match as text, so an inner join silently drops those rows — here **537 order lines carrying ₹91.7 lakh (1.99% of gross) across 94 distinct customers** — and nothing in the output indicates anything is missing.

Measured, including the fix:

```
print('order code lengths :', dict(body['customer_code'].str.len().value_counts().sort_index()))
print('master code lengths:', dict(cust['customer_code'].str.len().value_counts()))

master = set(cust['customer_code'])
fail = ~body['customer_code'].isin(master)
gross = qty * p_clean

print(f'rows failing the join : {fail.sum()}')
print(f'distinct codes lost : {body["customer_code"][fail].nunique()}')
print(f'gross they carry : {gross[fail].sum():.0f} ({gross[fail].sum() / gross.sum() * 100:.2f}%)')
print(f'after zfill(4) : {(~body["customer_code"].str.zfill(4).isin(master)).sum()} failures')
```

```
order code lengths : {2: 40, 3: 497, 4: 25432}
master code lengths: {4: 5027}
rows failing the join : 537
distinct codes lost : 94
gross they carry : 9,172,850 (1.99%)
after zfill(4) : 0 failures
```

The length distribution is the diagnosis and the fix in one output. The master is uniformly four characters; the order file has 40 two-character and 497 three-character codes. Those 537 rows are exactly the ones that lost leading zeros, and `zfill(4)` takes the failure count to **zero** — not "fewer", zero. A fix that resolves a join to exactly zero unmatched rows is a fix you can defend.

Why this is the most dangerous bug class in the chapter. Compare it to the others: a bad date produces a visibly odd chart, a bad category produces an unfamiliar label, a bad price produces a total someone may question. A broken join produces a **smaller, entirely plausible dataset**. There is no artifact. The only way to catch it is to measure the match rate deliberately, before and after.

Tier	What to say
Passes	Spots the leading-zero problem and pads the shorter side
Strong	Diagnoses it from the length distribution, quantifies rows and revenue lost, and verifies the fix reaches zero unmatched
Extra points	+ [+Validate] measure the match rate <i>before</i> joining and assert it, so a future file with a new key problem fails loudly instead of returning a short answer + [+Business] quantify in money, not rows: "₹91.7 lakh of revenue would silently vanish" is the version that gets the upstream export fixed + [+Edge cases] pad the <i>order</i> side up to four rather than stripping the master's zeros, because stripping could collide two distinct customers whose codes differ only by a leading zero + [+Trade-offs] the real fix is upstream: the export should quote the column or the key should be a type that cannot lose characters

Likely follow-ups: How would you verify that 0124 and 124 are genuinely the same customer? What if padding created a collision?

Red flag: casting both sides to integer, which "fixes" the match while destroying the key's actual format and risking collisions.

Learn it in: Chapter 14, §14.10 (repairing join keys); Chapter 13, §13.3 (join types).

Q72B-058 · How do you measure a match rate before you join, and what do you do with the answer?

Level: Mid · **Roles:** DA, DE, AE, BA

Remember it as: *Count the overlap before you join, not the rows after. A join tells you what matched; only a match rate tells you what did not.*

Answer in one line: Count how many left-side keys appear on the right (and the reverse), as rows and as a share of value, then compare that against an expectation you stated in advance — and investigate the gap rather than proceeding with whatever the join returns.

The pattern, measured both ways:

```
orders_key = body['customer_code'].str.zfill(4)

print(f'order lines whose customer exists : {orders_key.isin(master).mean() * 100:.2f}%')
print(f'customers with no orders this quarter: {(~cust["customer_code"].isin(set(orders_key))).sum()}')
```

```
order lines whose customer exists : 100.00%
customers with no orders this quarter: 790
```

Two numbers, two completely different meanings, and this is the point of the question.

- **100% of order lines match a customer.** That is a hard requirement: an order from a customer who does not exist is referential nonsense, so anything below 100% is a defect.
- **790 customers placed no orders.** That is not a defect at all. It is a business fact — 16% of the customer base was inactive in Q4 — and "fixing" it would be absurd.

A candidate who reports both numbers as "match rates" has missed that one side is a validation rule and the other is a finding. Knowing which is which requires understanding the relationship, not the SQL.

Tier	What to say
Passes	Uses <code>how='left'</code> and counts the nulls afterwards
Strong	Measures the overlap in both directions before joining, and distinguishes the direction that must be 100% from the direction where a gap is information
Extra points	+ [+Validate] state the expected rate before measuring, because a number with no expectation cannot be wrong + [+Business] measure the gap in revenue as well as rows, since 2% of rows can be 20% of value if the unmatched customers are large + [+Edge cases] <code>validate='m:1'</code> on a pandas merge raises if the right side is not unique, which catches the row-multiplication bug in Q72B-059 before it happens + [+Signpost] <code>indicator=True</code> labels each row <code>left_only</code> , <code>right_only</code> or <code>both</code> , which turns the match rate into an inspectable column

Likely follow-ups: What match rate would you accept? How would you investigate the unmatched rows?

Red flag: joining first and reporting the resulting row count as if it were the answer.

Learn it in: Chapter 14, §14.10; Chapter 13, §13.6.

Q72B-059 · You join orders to customers and get more rows than you started with. What happened?

Level: Mid · **Roles:** DA, DS, DE, AE

Remember it as: *A join multiplies. If the right side has duplicate keys, every duplicate fans out into extra rows — and every total built on it is inflated.*

Answer in one line: The right-hand table has duplicate keys, so each left row matched more than once and the row count multiplied — which also multiplies every revenue figure, making the total wrong in a way that looks like growth.

Why this file is a live trap. The CRM has **48 duplicate customer records** (Q72B-014). Join on `customer_code` before de-duplicating the master and every order line belonging to one of those customers appears twice, with its revenue counted twice.

The guard, which is one keyword:

```
dupe_master = cust['customer_code'].duplicated().sum()
print('duplicate codes in master:', dupe_master)

joined = body.merge(cust, on='customer_code', how='left', validate='m:1')
```

```
duplicate codes in master: 0
```

On this file `customer_code` happens to be unique — the 48 duplicates are duplicate *names* under different codes, which is a separate problem — so the join is safe and `validate='m:1'` passes silently. **That is exactly how a guard should behave:** invisible when things are fine, loud when they are not. Had the master contained a repeated code, `validate='m:1'` would raise `MergeError` instead of quietly returning an inflated table.

The habit worth stating: check the row count before and after every join, and assert the relationship you expect. A many-to-one join must never increase the row count.

```
assert len(joined) == len(body), f'join changed row count: {len(body)} -> {len(joined)}'
```

Tier	What to say
Passes	Identifies duplicate keys on the right side as the cause
Strong	Names the cause, adds <code>validate='m:1'</code> as the preventive guard, and asserts the row count across the join
Extra points	+ [+Validate] the row-count assertion catches every variant of this bug, including ones <code>validate</code> does not cover + [+Business] a fan-out inflates revenue, so it presents as unexplained growth — the most believable and therefore most dangerous kind of wrong number + [+Edge cases] the opposite failure is quieter still: an inner join that <i>reduces</i> the count, which looks like nothing at all happened + [+Clarify] decide what a duplicate master record should mean before joining — merge them, pick one by a rule, or escalate — because the join is not where that decision belongs

Likely follow-ups: What does `validate='1:1'` do differently? How would you find which keys fanned out?

Red flag: accepting a post-join row count without comparing it to the pre-join count.

Learn it in: Chapter 14, §14.10; Chapter 13, §13.3 and §13.6; Chapter 72, §72.7.

Rapid-fire, 72B.7

Roles: DA, DS, DE, AE and BA for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q72B-060	Why is an inner join dangerous on messy data?	Because it silently discards anything that does not match on both sides, so a broken key produces a smaller, entirely plausible result with no indication that rows were dropped	[+Validate] use a left join plus <code>indicator=True</code> while cleaning, and switch to inner only once the match rate is proven	Fresher · 13.3, 14.10
Q72B-061	What does <code>indicator=True</code> give you?	A <code>_merge</code> column labelling each row <code>left_only</code> , <code>right_only</code> or <code>both</code> , which turns "did the join work" from a guess into a value you can count and filter on	[+Business] <code>left_only</code> rows are usually the finding worth reporting	Mid · 13.6
Q72B-062	Trailing spaces in a join key — what happens?	The join silently fails for those rows, because <code>'0124 '</code> and <code>'0124'</code> are different strings; this is Q72B-037's invisible whitespace in its most expensive form	[+Validate] strip every key column before joining, unconditionally	Fresher · 14.8, 14.10
Q72B-063	Can you join on a float?	You should not: floating-point equality is unreliable, so matching on a float key can fail for values that are mathematically equal; cast to a string or integer key first	[+Edge cases] this is how a key that was promoted to <code>float64</code> by a blank value (Q72B-054) starts failing joins	Mid · 14.10, 17.6
Q72B-064	What is a composite key and when do you need one?	A key made of more than one column, needed when no single column is unique — a date plus a branch plus a product, say — and every column of it must be cleaned and type-matched on both sides or the join fails for the rows where any one differs	[+Trade-offs] a surrogate key avoids the fragility but has to be generated and maintained	Mid · 28.5
Q72B-065	Case sensitivity in a join key?	Databases differ — PostgreSQL compares strings case-sensitively while MySQL's default collation does not — so the same join can match in one engine and not the other, which is why keys should be normalised to one case before joining	[+Edge cases] this makes a migration between engines silently change results	Senior · 12.6, 14.10
Q72B-066	How do you find out <i>which</i> rows a join dropped?	Left-join with <code>indicator=True</code> and filter to <code>left_only</code> , then look at the actual key values — which is how you discover that the problem is leading zeros rather than genuinely missing customers	[+Signpost] this is the step that turned Q72B-057 from "537 rows missing" into a one-line fix	Mid · 13.6, 14.10

72B.8 Missing values, and the checks that must return zero

Q72B-067 · 14 rows have a blank quantity and 8 have a blank product ID. What do you do with each?

Level: Mid · **Roles:** DA, DS, DE, AE, BA

Remember it as: *There is no general rule for missing data. There are four actions, and the column decides which.*

Answer in one line: Treat them separately: a blank quantity cannot be invented, so those rows are quarantined; a blank product ID can be **repaired from evidence** here, because `unit_price` identifies the product — and both decisions get logged.

The four actions, which is the framework worth having ready:

Action	When	Here
Standardise	The absence is real and meaningful	The CRM's 100 missing cities, recorded as blank, N/A, -, unknown, NULL → one representation
Repair	An independent source gives the value	8 blank <code>product_id</code> , recoverable from <code>unit_price</code>
Quarantine	The value matters and cannot be recovered	14 blank <code>quantity</code> — no quantity, no revenue, no guess
Keep and label	The absence is itself information	150 customers with no email: that is a CRM completeness fact, not an error

Why `product_id` is repairable — checked, not assumed. The claim is that price identifies product, and that is a property to verify before relying on it:

```
known = body[body['product_id'] != '']
pairs = known.groupby(p_clean[body['product_id'] != ''])['product_id'].unique()
print({int(k): v[0] for k, v in pairs.items()})
print('one-to-one:', all(len(v) == 1 for v in pairs))
```

```
{115: '103', 290: '108', 380: '107', 430: '101', 620: '104', 750: '102', 1400: '105'}
one-to-one: True
```

Each of the seven prices maps to exactly one product, so a blank product with a price of 430 can only be product 101. **That is evidence, and the one-to-one check is what makes it evidence rather than an assumption** — had any price mapped to two products, the repair would have been invalid and those 8 rows would go to quarantine instead. The repair still gets a `product_repaired` flag, for the same reason the date repair does.

Why `quantity` is not. There is no second source for how many units were ordered. Imputing the median would manufacture revenue that never existed, on a financial table. These 14 rows are set aside, counted, and reported.

Tier	What to say
Passes	Suggests dropping the rows, or filling with a median or zero
Strong	Distinguishes the four actions and applies a different one to each column, with the reason tied to whether an independent source exists
Extra points	+ [+Validate] every repaired value carries a flag column, so a later reader can separate observed data from inferred data + [+Business] on a revenue table, imputation is manufacturing money; quarantine-and-report is the only defensible choice + [+Edge cases] the price-to-product mapping must be one-to-one for the repair to be valid, which is a property to check rather than assume + [+Trade-offs] quarantining 14 of 25,832 rows changes no total materially, so the cost of the conservative choice here is nil — which is worth saying, because conservatism is not always free

Likely follow-ups: What if 2,000 rows had a blank quantity instead of 14? When is imputation acceptable?

Red flag: one blanket `.fillna()` across the whole DataFrame.

Learn it in: Chapter 14, §14.5 (the four actions) and §14.12.

Q72B-068 · Write the validation rules for this cleaned table. What must return zero?

Level: Mid · **Roles:** DA, DE, AE, BA

Remember it as: A validation rule is a query that must return no rows. If it returns one, the pipeline stops.

Answer in one line: Rules expressed as counts that must be zero — no null keys, no unmapped categories, no unparsed dates, no values outside known sets, no duplicate primary keys, no quantity above the historical maximum — asserted in code so a violation fails the run instead of reaching a report.

The suite for this table, each rule with the defect it exists to catch:

```
checks = {
    'primary key not unique'      : clean['order_item_id'].duplicated().sum(),
    'null primary key'           : clean['order_item_id'].isna().sum(),
    'status unmapped'            : clean['status'].isna().sum(),
    'branch unmapped'            : clean['branch'].isna().sum(),
    'date null and not repaired' : (clean['order_date'].isna() & ~clean['date_repaired']).sum(),
    'date outside Q4 2025'       : (~clean['order_date'].dt.to_period('M')
                                     .astype(str).isin(['2025-10', '2025-11', '2025-12'])).sum(),
    'price not a known price'     : (~clean['unit_price'].isin([115, 290, 380, 430, 620, 750,
1400])).sum(),
    'discount not a known tier'   : (~clean['discount_pct'].isin([0, 5, 8, 10, 12])).sum(),
    'quantity above historic max' : (clean['quantity'] > 90).sum(),
    'mixed quantity units'        : (clean['qty_unit'].nunique() - 1),
    'customer not in master'      : (~clean['customer_code'].isin(master)).sum(),
    'text with stray whitespace' : sum((clean[c] != clean[c].str.strip()).sum()
                                     for c in ['status', 'branch', 'sales_rep']),
    'null product_id'            : clean['product_id'].isna().sum(),
}
for name, n in checks.items():
    print(f'{"PASS" if n == 0 else "FAIL"} {name:<30} {n}')
assert all(v == 0 for v in checks.values()), 'validation failed'
```

Run against the fully cleaned table — dates dispatched by shape, prices detokenised, discounts normalised, categories mapped, keys zero-padded, duplicates removed and the 20 bad-quantity rows quarantined — all thirteen return zero:

```
PASS primary key not unique      0
PASS null primary key            0
PASS status unmapped             0
PASS branch unmapped             0
PASS date null and not repaired   0
PASS date outside Q4 2025        0
PASS price not a known price      0
PASS discount not a known tier    0
PASS quantity above historic max 0
PASS mixed quantity units        0
PASS customer not in master      0
PASS text with stray whitespace  0
PASS null product_id             0
```

The price not a known price rule is in this chapter because it failed. On the first run of this suite it returned **1,197** — which is how the Rs. 1,400 → 0.14 regex bug in Q72B-045 was found. A null check would not have caught it, because 0.14 is a perfectly good number. Only the set-membership rule did.

Notice what makes these rules good, because that is the real question. Each one is:

- **Specific.** "Price is one of seven known values" is enormously stronger than "price is positive". A corrupted price of 431 passes the weak rule and fails the strong one.
- **Zero-valued.** There is no threshold to argue about and no judgement at review time.

- **Traceable to a defect you actually found.** Every rule above exists because this chapter measured the corresponding problem. That is how a validation suite should be built — from the profile (Q72B-004), not from a generic checklist.

Tier	What to say
Passes	Lists sensible checks: nulls, duplicates, ranges
Strong	Expresses them as must-be-zero counts asserted in code, and derives each from a defect found during profiling
Extra points	+ [+Validate] prefer set membership over range checks where the set is known, because it is far tighter + [+Business] print the full pass/fail table rather than only the failure, so a successful run leaves positive evidence someone can read + [+Edge cases] <code>assert</code> is removed entirely when Python runs under <code>-O</code> , so a production pipeline should raise an explicit exception rather than rely on <code>assert</code> + [+Trade-offs] hard-fail suits a nightly batch; a daily report that must ship may need quarantine-and-continue with the counts on the output

Likely follow-ups: Which of these would you run in SQL instead? How do you stop the suite becoming unmaintainable?

Red flag: validation as a notebook cell someone reads, rather than an assertion that fails the run.

Learn it in: Chapter 14, §14.12 (rules that must return zero); Chapter 46, §46.4.

Q72B-069 · Your cleaned total is ₹44.38 crore and the truth file says ₹44.26 crore — within 0.28%. Is your pipeline correct?

Level: Senior · **Roles:** DA, DS, AE, BA

Remember it as: *A number that is nearly right is not evidence of a right method. Errors in opposite directions cancel, and the total says nothing about either.*

Answer in one line: **No.** That figure still contains three uncorrected defects — 137 duplicate lines, 6 extra-zero quantities and 141 unconverted carton rows — and the reason the total looks right is that two of them inflate it while the third understates it by almost the same amount.

The decomposition, measured. Prices and discounts cleaned, nothing else fixed:

	Effect on the total
137 duplicate lines, still present	+₹0.25 cr (inflates)
6 extra-zero quantities (700 for 70)	+₹0.11 cr (inflates)
141 carton rows, 1 CTN = 10 PCS	−₹0.22 cr (understates)
Net effect of all three	+₹0.14 cr
Actual gap to truth	+₹0.12 cr

₹0.36 crore of inflation against ₹0.22 crore of understatement. Three real defects, each worth between a tenth and a quarter of a crore, and they sum to a 0.28% discrepancy that any reviewer would wave through. (The

remaining ₹0.02 crore is the 14 blank-quantity rows, which the truth file quarantines and this total still carries.)

What a correct pipeline gives. De-duplicate, repair the six quantities, convert the cartons at 10:

```
correct pipeline : 44.24 cr
truth, all lines : 44.26 cr
difference       : -1.90 lakh (-0.043%)
```

Now the agreement means something, because the method is right **and** the residual is explained — it is the handful of quarantined rows, named and counted.

This is the discipline the question is testing. Reconciliation is only evidence when three things hold:

1. **The definitions match.** Same rows, same filters, same grain. "Revenue" here is at least four different numbers — all lines (₹44.26 cr), non-cancelled (₹42.39 cr), gross before discount (₹46.01 cr), and net of quarantine — and comparing any two of them proves nothing. Picking the wrong baseline is the most common version of this mistake: compare that same ₹44.38 crore against the **non-cancelled** total of ₹42.39 crore and you would conclude the pipeline overstates by 4.7%, which is equally meaningless. 2. **You stated the expected figure first.** A target found after the fact is a number you steered toward. 3. **The residual is explained and named**, not absorbed by a tolerance — ₹1.90 lakh of quarantined rows, not "close enough".

A candidate who says "0.28%, good enough" has made the most expensive mistake in analytics: treating agreement as correctness.

Tier	What to say
Passes	"I would check which rows are included" — the right instinct, stopping short of the work
Strong	Refuses to accept the total as evidence, lists what is still uncorrected, and recognises that opposing errors can cancel
Extra points	+ [+Validate] decompose the residual into named, quantified causes, which is the only thing that distinguishes a correct pipeline from a lucky one + [+Business] publish the definition with the number — "net revenue, all lines including cancelled" — because most reconciliation disputes are definition disputes wearing a numerical disguise + [+Edge cases] errors cancelling is not rare: any pipeline with one inflating and one deflating defect will produce a plausible total, which is why a single matching figure is weak evidence + [+Trade-offs] a tolerance is sometimes necessary, but it is stated in advance with a reason, not discovered to have been satisfied

Likely follow-ups: How would you prove each component of that decomposition? What tolerance would you accept, and why?

Red flag: accepting a close match as proof, or comparing two totals without checking they count the same rows under the same definition.

Learn it in: Chapter 14, §14.13 (reconciling to the rupee); Chapter 10, §10.6 (defining a metric).

Rapid-fire, 72B.8

Roles: DA, DS, DE, AE and BA for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q72B-070	What belongs in a cleaning log?	Every decision and its reason: the rule applied, the rows affected, the source of any repair, who approved a business rule, and the date — so a number can be defended months later without re-deriving it	[+Business] the log is what converts "trust me" into "here is why"	Mid · 14.14
Q72B-071	Why flag repaired values rather than just fixing them?	Because an unflagged repair is indistinguishable from observed data, so nobody downstream can exclude inferred values from an analysis that should not use them	[+Validate] <code>date_repaired</code> and <code>product_repaired</code> on this table are the pattern	Mid · 14.7, 14.14
Q72B-072	Clean in SQL or in pandas?	Wherever the data already is and wherever the result must live: SQL when the source is a database and the cleaning must be repeatable and set-based; pandas for exploratory profiling and file-based inputs — the logic is identical either way, as Chapter 14 shows by doing both	[+Trade-offs] SQL wins on volume and auditability, pandas on iteration speed	Mid · 14.13

72B.9 The full rapid-fire round

Every row here is a question that has been asked in a real data interview. Roles: DA, DS, DE, AE and BA throughout unless narrowed.

#	Question	One-line answer	Extra point	Level · learn it in
Q72B-073	What is the difference between data cleaning and data wrangling?	Cleaning fixes what is wrong — duplicates, bad types, inconsistent categories; wrangling reshapes what is right into the form an analysis needs, such as pivoting long to wide or joining sources	[+Clarify] most job ads use the two interchangeably, so ask what the role actually involves	Fresher · 14.1
Q72B-074	What is tidy data?	One row per observation, one column per variable, one table per kind of observation — the shape that makes grouping, joining and plotting straightforward instead of requiring a reshape first	[+Signpost] "one row per observation" is the grain question of Q72B-005	Fresher · 25.12
Q72B-075	Long or wide format — which and when?	Long for storage, analysis and anything with a varying number of categories; wide for human reading and for a few fixed categories — convert at the presentation layer, not in the warehouse	[+Trade-offs] wide tables break every time a new category appears	Mid · 25.12
Q72B-076	<code>melt</code> and <code>pivot</code> — what do they do?	<code>melt</code> turns columns into rows (wide to long) and <code>pivot</code> turns rows into columns (long to wide); <code>pivot_table</code> is <code>pivot</code> with an aggregation, for when the combination is not unique	[+Edge cases] plain <code>pivot</code> raises on duplicate index/column pairs, which is a useful duplicate detector	Mid · 25.12
Q72B-077	What is the first thing you check in a file you have been given?	Whether the row count and grain are what you were told, because everything downstream depends on it and it is the assumption most often wrong	[+Signpost] Q72B-005 and Q72B-015	Fresher · 14.3
Q72B-078	How do you handle an encoding error on read?	Identify the real encoding rather than suppressing the error: try <code>utf-8</code> , then <code>utf-8-sig</code> for a BOM, then <code>cp1252</code> / <code>latin-1</code> for Windows exports — and never pass <code>errors='ignore'</code> , which silently deletes characters including ones inside keys	[+Edge cases] <code>errors='replace'</code> corrupts a join key into an unmatched value	Mid · 14.3
Q72B-079	A column is 95% null. Keep it or drop it?	Ask what the 5% means before deciding: a sparse column can be the most valuable one in the table (a churn reason, a complaint code), and 95% null may be correct for a field that only applies in rare cases	[+Business] nullness is a property of the process, not a defect to threshold	Mid · 14.5
Q72B-080	What is referential integrity and how do you check it without a database?	Every foreign key must exist in its parent table; check it with a set membership test — <code>~orders.customer_code.isin(set(customers.customer_code))</code> must be empty	[+Validate] this is Q72B-058's 100% requirement as a validation rule	Mid · 12.5, 14.10
Q72B-081	58 email addresses have no <code>@</code> . Reject, fix or keep?	Keep and flag rather than reject, because the record is still a real customer; mark the email unusable so it is excluded from any send, and report the count upstream as a form-validation gap	[+Business] deleting the record loses a customer; deleting the email loses the evidence	Fresher · 14.5

#	Question	One-line answer	Extra point	Level · learn it in
Q72B-082	Five signup dates are in the 2060s. What do they tell you?	That the entry form has no upper bound, which is a validation gap upstream; the dates themselves are unrecoverable without a second source, so they are quarantined, not guessed	[+Edge cases] a two-digit year entry (60 meaning 1960) is a common cause worth checking before quarantining	Mid · 14.7
Q72B-083	Should cleaning be idempotent?	Yes: running it twice on the same input must give the same output, which means never mutating the source in place and never applying a transformation that is not safe to repeat — stripping twice is harmless, multiplying a discount by 100 twice is not	[+Signpost] Q72B-046's fraction fix is not idempotent unless the condition is re-checked	Senior · 46.4
Q72B-084	Where does cleaning belong in a pipeline?	In a defined layer between raw and modelled data, with the raw data preserved unchanged, so cleaning can be re-run, corrected and audited without re-fetching the source	[+Business] never overwrite raw: it is the only thing you cannot reconstruct	Mid · 45.2, 49.5
Q72B-085	Why keep the raw file after cleaning?	Because every cleaning decision may turn out wrong, and without the raw input you cannot re-derive the correct answer or prove what the source actually said	[+Validate] reconciliation needs the source; without it there is nothing to reconcile to	Fresher · 14.13
Q72B-086	What is a data contract and how does it prevent this chapter?	An agreement with the producer on schema, types, formats and cadence, so a date format change or a new status value is a breach to be fixed at source rather than a surprise to be cleaned downstream	[+Signpost] every defect in this chapter is a contract that was never written	Senior · 47.7
Q72B-087	How would you clean 500 GB that does not fit in memory?	Push the work to where the data is — SQL in the warehouse, or a chunked/lazy engine such as Polars, DuckDB or Spark — and profile on a sample while validating on the whole, because the rules are the same and only the execution changes	[+Trade-offs] sampling for profiling is fine; sampling for validation is not	Senior · 49.7, 50.2
Q72B-088	How do you make cleaning reproducible?	Scripted, not manual: version-controlled code, pinned library versions, mapping tables as files, no hand edits to data, and a logged run that can be repeated to the same output	[+Business] a manual Excel clean cannot be audited or repeated, which is why it fails the moment someone asks how a number was produced	Mid · 14.14, 46.2
Q72B-089	Your cleaning drops 3% of rows. Ship it?	Not until the 3% is explained and categorised: 3% of junk rows is fine, 3% of the largest customers is not, so the question is what those rows are and what value they carry, not what the percentage is	[+Validate] Q72B-015's reconciliation is how you answer	Mid · 14.13
Q72B-090	Who owns data quality?	The producer owns correctness at source and the consumer owns validation on arrival — because a consumer who only cleans is permanently patching, and a producer with no feedback never learns what is broken	[+Business] every defect in this chapter should go back upstream as a report, which is the only fix that scales	Senior · 47.7

#	Question	One-line answer	Extra point	Level · learn it in
Q72B-091	What would you automate first?	The validation rules, not the fixes: automated checks tell you when something changed, which is the expensive thing to discover late, whereas an automated fix for an unvalidated problem can corrupt data at scale	[+Trade-offs] automated cleaning without automated validation increases blast radius	Senior · 14.12, 46.4
Q72B-092	A stakeholder says last month's number has changed. How do you answer?	With the cleaning log and the reconciliation: which rule changed, how many rows it affected, and what the before and after totals were — a specific, auditable answer rather than a reassurance	[+Business] this single capability is most of what earns an analyst trust	Mid · 14.14, 46.5
Q72B-093	What is the most common cleaning mistake you have seen?	Fixing data without measuring the fix — no before-and-after counts, no reconciliation, no log — so the pipeline produces a plausible number nobody can defend and nobody can reproduce	[+Signpost] every section of this chapter is one instance of that mistake	Mid · 14.14
Q72B-094	When is data too dirty to use?	When the defects reach the measure you need and cannot be repaired from any independent source — then the honest answer is to report what cannot be answered and what it would take to fix the source, not to produce a number with a caveat nobody will read	[+Business] saying "this data cannot answer that question" is a senior act, not a failure	Senior · 14.12, 10.7
Q72B-095	How long should cleaning take?	Longer than stakeholders expect and the question is usually misframed: a one-off clean is hours, but a repeatable, validated, logged cleaning layer is days and pays for itself the second time the file arrives	[+Business] quote the repeatable version, and say why	Mid · 14.1
Q72B-096	<code>apply</code> with a cleaning function, or vectorised string methods?	Vectorised <code>.str</code> methods, which are both much faster and easier to read; <code>apply</code> runs a Python function per row and is the slowest option, worth it only for logic that genuinely cannot be vectorised	[+Trade-offs] correctness first: a readable <code>apply</code> beats an unreadable vectorised chain if the data is small	Mid · 25.7, 72.5
Q72B-097	How do you clean a free-text field?	Decide first whether you need it as a category or as text: as a category it needs normalisation plus a mapping table plus a human-reviewed long tail; as text it needs only whitespace and encoding fixes — and conflating the two is how a field ends up with 400 categories	[+Clarify] ask what question the field is meant to answer	Mid · 14.8
Q72B-098	What is schema-on-read, and what does it cost?	Storing raw data and applying structure at query time, which is flexible and cheap to ingest but moves every cleaning decision to every consumer — so the same file gets cleaned five different ways and five different numbers get reported	[+Business] a shared cleaned layer exists to prevent exactly that	Senior · 49.5
Q72B-099	Would you ever ship a report from data you know is dirty?	Yes, with the defects quantified and stated on the output, when a timely approximate answer beats a late exact one — but never silently, and never without saying which direction the error runs	[+Business] "revenue is understated by roughly 2% because 537 lines could not be matched" is a usable, honest number	Senior · 10.7, 14.12

#	Question	One-line answer	Extra point	Level · learn it in
Q72B-100	What single habit would most improve a team's data quality?	Reconciling every output to its source and publishing the difference, because it makes every silent error in this chapter loud, and it is the one check that catches problems nobody anticipated	[+Signpost] Q72B-015 and Q72B-069 are the two halves of it	Senior · 14.13

72B.10 Reproduce every number in this chapter

Nothing in this chapter is an estimate, and you should not take that on trust. Every figure came from the two files in `companion/ch14/`, and Chapter 14 also ships the correct answer, so you can check both the problems and the solutions.

File	What it is
<code>orders_q4_2025_export.csv</code>	The messy export. 25,976 rows below the header
<code>customers_crm_export.csv</code>	The messy customer master. 5,027 records
<code>clean_truth_orders_q4_2025.csv</code>	The right answer. 25,832 clean order lines
<code>answer_key.json</code>	The planted defect counts
<code>status_map.csv</code> , <code>branch_map.csv</code> , <code>city_map.csv</code>	Starter mapping tables, incomplete on purpose
<code>clean_orders_pandas.py</code>	Chapter 14's full cleaning script
<code>ch14_queries_postgresql.sql</code> , <code>ch14_queries_mysql.sql</code>	The same cleaning in SQL

The setup every snippet in this chapter assumes:

```
import pandas as pd

raw = pd.read_csv('orders_q4_2025_export.csv', dtype=str, keep_default_na=False)
cust = pd.read_csv('customers_crm_export.csv', dtype=str, keep_default_na=False)

body = raw[(raw['order_item_id'] != 'order_item_id') & (raw['order_item_id'] != '')].copy()
s = body['order_date']

qty = pd.to_numeric(body['quantity'], errors='coerce')
p_clean = pd.to_numeric(body['unit_price']
                        .str.replace(r'(?i)^\r?\s*', '', regex=True)
                        .str.replace('₹', '', regex=False)
                        .str.replace(',', '', regex=False), errors='coerce')
master = set(cust['customer_code'])
```

Two notes on the counts, so the numbers in this chapter line up with Chapter 14's.

Most figures here are quoted on `body`: **25,969 rows**, which is the file minus the 6 repeated header rows and the 1 footer row, but **still containing the 137 duplicates**. Chapter 14's answer key and truth file are quoted on the **25,832** de-duplicated lines. Both are correct for their stage, and where a figure could be read either way this chapter says which one it is. That is Q72B-015's discipline applied to the chapter itself.

Where a measurement here differs slightly from `answer_key.json`, the measurement is what the file actually contains, and the difference is the duplicates. Three worth naming: this chapter measures **2,316** currency-text

prices (key: 2,300), **1,371** fractional discounts (key: 1,362) and **1,604** rows whose UTC date differs from the IST date (key: 1,598). The key records what the generator planted; the body contains those rows plus their duplicates. The `codes_without_leading_zeros` entry is a different case — the key records 3,210, which is the number of Kolkata-branch lines, but only the **537** of them whose code actually begins with a zero lose a character and fail the join. Where the two disagree, this chapter uses the measurement and says so.

Note the price cleaner: it removes the known currency tokens rather than keeping a character class, for the reason Q72B-045 measures. Substituting `[\^0-9.]` there changes the gross total by ₹2.13 crore.

Verified on: Python 3.12.0, pandas 3.0.2, numpy 2.4.3. The outputs in this chapter do not depend on those versions, with one exception that is itself a question: the string-inference behaviour in Q72B-002 changed in pandas 3.0, and that question says so.
